

5 Services

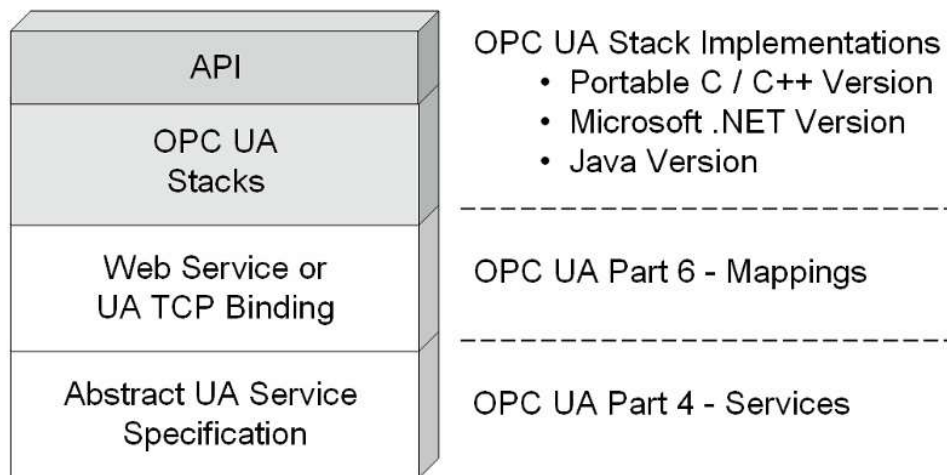
5.1 Overview

The OPC UA Services are defining the data communication on application level. Services are methods used by an OPC UA client to access the data of the Information Model provided by an OPC UA server. Similar to Classic OPC, where the OPC specifications just defined Application Programming Interface (APIs) between applications, the Services define the communication interface between UA applications. The definition of the Services is independent of the transport protocol and the programming environment that is used to develop an OPC UA application. This is the fundamental difference to Classic OPC where the definition of the APIs was bound to a specific transport mechanism – Microsoft Component Object Model (COM).

The independence of the transport protocol and the programming environment requires an abstract definition of the Services. This abstract definition [UA Part 4] can be applied to different transport mechanisms (see Chap.6) defining the representation of the Services on the wire [UA Part 6] and to different implementations of the transport mechanisms in OPC UA Stacks (see Sect. 6.5) in different programming languages. The language specific APIs for the application development are defined by the OPC UA Stacks based on the abstract UA Service definition. The different communication layers are shown in Fig. 5.1.

Like Classic OPC, the OPC UA Services are designed for exchanging bulk data between UA applications running in different processes or on different network nodes to reduce the roundtrips between the applications. For example, a Read method does not read one Variable but allows defining a list of Variables to read. OPC UA Services are reduced to a generic set of methods. There are two main reasons for this reduction. On the one side a lot of information provided in Classic OPC is now modeled in the server Address Space and accessed with a generic method like read instead of having specific methods to access the information. A simple example is the server status information. In Classic OPC, this information was obtained by a special GetStatus Method. In OPC UA, this information is modeled as server status Variable and can be accessed with the Read Service or monitored for changes. Another reason was the design goal to provide only one generic Service for a specific functionality and not a long list of specialized methods for different variations of information.

This chapter describes the general Service behavior and the functionality provided by the different Services. They are partitioned into Service Sets in the corresponding OPC UA specification [UA Part 4]. The grouping in this chapter is based on use cases summarized in Table 5.1 and does not exactly match the Service Set partitioning. Each Service is described in this chapter with its key parameters. A complete list of parameters and their detailed description can be found in [UA Part 4] or in API documentations.

**Fig. 5.1** OPC UA communication layering**Table 5.1** Services grouped by use cases

Use case	Service sets or services
Find servers	Discovery Services Set
Connection management between clients and servers	Secure Channel Service Set Session Service Set
Find information in the Address Space	View Service Set
Read and write data and metadata	Read and Write Service
Subscribe for data changes and Events	Subscription Service Set Monitored Item Service Set
Calling Methods defined by the server	Call Service
Access history of data and Events	HistoryRead and HistoryUpdate Service
Find information in a complex Address Space	Query Service Set
Modify the structure of the server Address Space	Node Management Service Set

5.2 General Service Concepts

The Service definition uses the request and response pattern known from Web Services where each Service is composed of a request and a response message. To invoke a Service in the server, the client sends a request message to the server. After processing the request, the server sends a response message back to the client. Since this message exchange is asynchronous, all Service invocations are asynchronous by definition. After sending the request message, the client application can process other functionality until the response message arrives. Most UA Stack APIs provide synchronous versions of the APIs for convenience. This is a general

enhancement compared to Classic OPC where all functions are synchronous and only a few functions were provided as asynchronous versions in addition.

5.2.1 Timeout Handling

The OPC UA data communication is designed for data exchange between different systems, typically running on different network nodes or in different processes. Especially network communication can be interrupted at any time and it is important to detect and handle these failures correctly.

An important concept in such an environment is configurable timeouts for Service invocations to get a timely detection of communication failure. For this reason each single Service call has individual timeouts defined by the client. This is an important enhancement over Classic OPC.

The communication stack on the client side returns the call to the API or sends the callback with a timeout status if the timeout expires. But the timeout set on the client-side UA stack is also sent to the server to detect Service invocations that are not longer expected to return to the client side.

5.2.2 Request and Response Headers

Each Service contains the same headers for request messages and for response messages. For this reason they are not listed for each Service in this chapter. They are containing common Service parameters like the token for the Session context or the result of the Service call. The request header parameters are described in Table 5.2 and response header parameters are described in Table 5.3.

Table 5.2 Request header parameters

Parameter	Description
AuthenticationToken	The secret Session identifier used to assign the Service call to the Session context created between client and server application
RequestHandle	A client defined handle assigned to the Service call
Timestamp	The time the client sent the request
TimeoutHint	The timeout set in the client side UA Stack for the call. This hint can be used by the server to cancel long running calls if the timeout expires
ReturnDiagnostic	Indicates if the client requests the server to return additional detailed diagnostic information in the case of an error instead of returning only a status code
AuditEntryId	A string that identifies the client or user that initiated the action. The string is empty if this parameter is not used. It is used for auditing (see Sect. 9.5)

Table 5.3 Response header parameters

Parameter	Description
ServiceResult	UA defined result code for the Service call
RequestHandle	The client defined handle assigned to the Service call
Timestamp	The time the server sent the response
ServiceDiagnostics	Client requested detailed diagnostic information

5.2.3 Error Handling

Error handling is an important part of the Service parameters and Service handling since communication between distributed systems from different vendors can cause errors on different levels and in different scenarios. Errors can happen in normal operation, for example, when a client is using wrong parameters like NodeIds of Nodes that are no longer available or they can happen in communication error scenarios when the connection between client and server or between server and the underlying system¹ is interrupted.

There are two types of error information used in the Services. The one type is an error code called *StatusCode*. The *StatusCode* is a 32-bit unsigned integer. The most significant 16 bits represent the numeric value of the code that shall be used for detecting specific errors or conditions. The least significant 16 bits are bit flags that contain additional information but do not affect the meaning of the *StatusCode*. The two most significant bits are indicating the overall severity which could be *Good* for success, *Uncertain* for warning, and *Bad* for failure. Status codes are only defined by OPC UA and cannot be extended by vendors or other organizations.

The second type of error information is the *DiagnosticInformation*. This structure contains additional information for a *StatusCode* including vendor-specific error codes, a localized description of the error, and a text field for additional information. The diagnostic information can be nested to be able to provide an error stack. There is a *DiagnosticInformation* field available for each *StatusCode* field in a Service, but the additional information is only returned by the Server if requested by the client. The *DiagnosticInformation* fields are not contained in the Service descriptions in this chapter.

The error information² is provided on two levels. The first level is the result of the Service call. The second level is the list of operations inside the Service call. Since OPC UA supports bulk operations for all Services used to exchange data, some operations in a Service call can fail while others succeed. An example is the Read Service where the client can read a list of Variable values in one Read Service call. Each Variable in the Read is an operation.

¹For example devices connected to the server by an interrupted network.

²Combination of *StatusCode* and *DiagnosticInformation*.

Clients must check the results always on both levels since both can fail. In the first step, the client must check if the Service call succeeded. If not, none of the result fields is valid. If the Service succeeded, each operation StatusCode must be verified by the client before using the result data of the operation.

Compared to Classic OPC, the error handling was simplified in OPC UA. Classic OPC provided a result code and a quality code for read and data change methods. Clients needed to check first the result code and then the quality code but only one field was able to contain error information. OPC UA provides only one StatusCode field which contains general error codes and also quality codes for values in the same field.

5.2.4 Extensible Parameters

Extensible parameters are used to add flexibility and variations to Services without having the need to extend the number of Services. Extensible parameters are used everywhere where a Service parameter can contain different structures for different use cases. Samples for the use of extensible parameters are the Subscriptions where different types of information like data and Events are handled with the same Services using extensible parameters for the filters and the transport of the notifications from server to client. Another sample is the history access Services where only two Services are defined for the read and update access to different type of data and Events.

5.2.5 Communication Context

OPC UA Services are not stateless and cannot be called without establishing a communication context on different levels. For this reason a lot of Services are not used for data transfer but to create, maintain, and modify these different levels of communication context illustrated in Fig. 5.2.

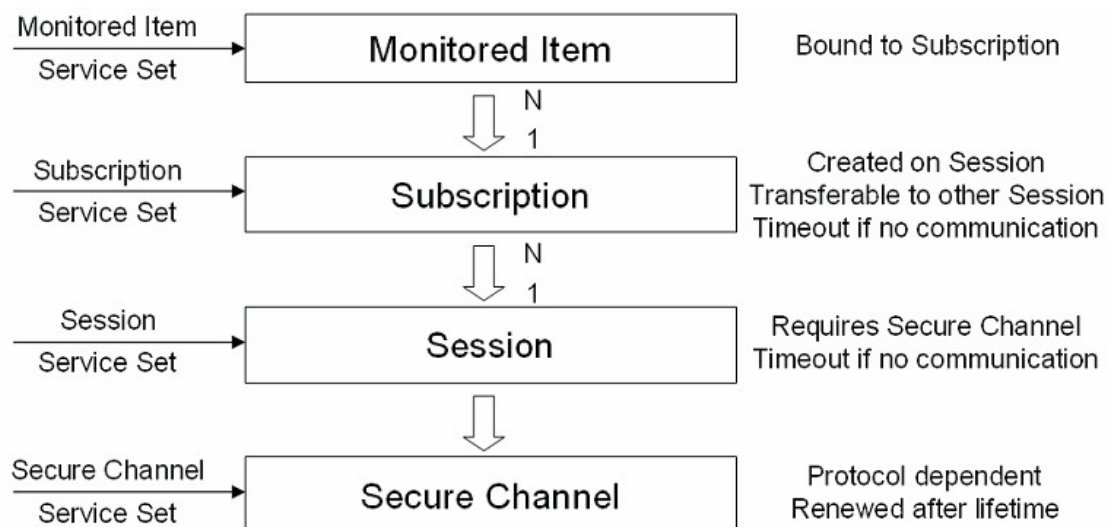


Fig. 5.2 Communication context

The Secure Channel is the low - level and protocol - dependent channel to secure the communication and the exchanged messages. This level is handled completely by the UA communication stacks hiding the different possible protocols. The Secure Channel must be renewed after the lifetime negotiated during the first establishment of the Secure Channel to reduce the risk of compromising the channel security.

The Session is the connection context between the two applications created on the top of and in the context of a Secure Channel. The lifetime of the Session is independent of the Secure Channel and another Secure Channel can be assigned to the Session. A Session has a timeout that allows the server to free the resources for a Session after a defined time period. The timeout gets reset with each Service invocation in the Session context received by the server.

Multiple Subscriptions can be created in the context of a Session. A Subscription is the context to exchange data changes and Event notifications between server and client. A Subscription requires a Session to transport the data to the client but it can be transferred to another Session for example to be used in a Session created by a redundant backup client if the client that created the Subscription is no longer available. Therefore the Subscription lifetime is independent of the Session lifetime and a Subscription has a timeout that gets reset every time data or keep-alive messages get sent to the client.

Multiple Monitored Items can be created in a Subscription but they are bound to this Subscription. A Monitored Item is used to define the Attribute of a Node that should be monitored for data changes or to define the Event source that should be monitored for Event notifications.

OPC UA defines 37 Services whereof 21 Services are used to manage the communication infrastructure and context and only 16 Services are used to exchange different types of information.

5.2.5.1 Services Used to Exchange Information

Table 5.4 provides a list of Services used to exchange information between OPC UA client and OPC UA server applications.

Table 5.4 Services used to exchange information between client and server

Service	Description
Browse BrowseNext	Navigate through the nodes of the OPC UA server address space. The client defines a starting node and filter criteria. The server returns a list of referenced nodes passing the filter
TranslateBrowsePaths- ToNodeIds	Get NodeIds of Object components based on the knowledge about the ObjectType
Read	Read Attributes of nodes including Values of Variables
Write	Write Attributes of nodes including Values of Variables

(Continued)

Publish Republish	Send changed data or Events for a Subscription from the OPC UA server to the OPC UA client
Call	Call a Method in the server
HistoryRead	Read the history of Variable Values or the history of Events
HistoryUpdate	Update the history of Variable Values or the history of Events
AddNodes	Adding nodes to the server address space. This includes the instantiation of Object instances
AddReferences	Adding references between nodes in the server address space
DeleteNodes	Deleting nodes in the server address space
DeleteReferences	Deleting references between nodes in the server address space
QueryFirst QueryNext	Returns a list of nodes and Attribute values based on complex filter criteria operating on the whole address space of a server

5.2.6 Conventions for Describing Services in this Chapter

The following sections are using tables to describe the Service parameters. Not all parameters are contained since some parameters like the diagnostic information are described as general concepts and other parameters are not important for understanding the Services. The data types of parameters are also not contained as general information in the tables. They are mentioned when it is important for understanding the Service. A complete list of parameters, data types, and their detailed description can be found in [UA Part 4] or in API documentations.

5.3 Finding Servers

The Discovery Service Set is used by clients to find available OPC UA servers and to get information about the available Endpoints of a server. It is used by the server to register with a Discovery server. The Discovery server maintains the list of available servers. The available Endpoints are provided by each server.

An Endpoint defines the used network protocol and the necessary security settings to be able to connect to the Endpoint of a server. Fig. 5.3 shows the interaction between UA client, Discovery server, and UA server.

Like all other Services, the Discovery Services can be invoked over the network and thus allow a network-wide discovery. Nevertheless OPC UA defines only the behavior of a local Discovery Server expecting that each node with more than one OPC UA server is running a Discovery Service. The definition of network-wide discovery will be added in a future release of OPC UA.

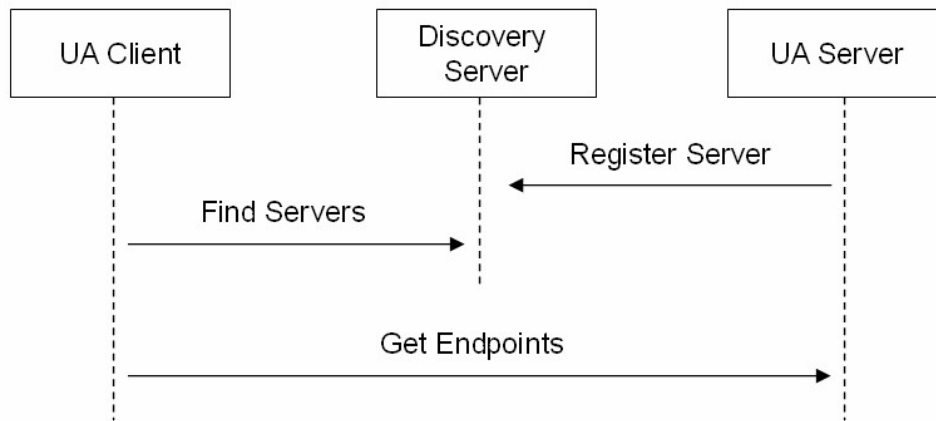


Fig. 5.3 Use of Discovery server and the Discovery Service Set

5.3.1 Service *FindServers*

The FindServers Service is implemented by the Discovery server returning a list of registered servers and by each UA server returning only itself. This feature is necessary to ensure the same behavior if only one server is running on a network node without an additional Discovery server. The parameters are described in Table 5.5.

This Service can be called on a Secure Channel without security enabled and without having a Session context created since this is not possible without the information returned during the discovery process.

Table 5.5 FindServers Service parameters

Request parameters		Description
LocaleIds []		A list of locales for the server name returned in the application description. The server should use the first locale in the list that it supports
ServerUris []		A list of server URIs to allow the client to request information about specific servers. All available servers are returned if an empty list is passed in
Response parameters		Description
Servers []		A list of application description structures for each returned server
	ApplicationUri	Globally unique identifier for the server instance
	ApplicationName	Human readable name for the server
	ProductUri	Globally unique identifier for the server product
	ApplicationType	The type of application which could be server, client, client and server and discovery server
	DiscoveryUrls []	The available URLs of the server that allow calling GetEndpoints without requiring a secure connection

5.3.2 Service *GetEndpoints*

This Service returns the Endpoints supported by the server implementing this Service. An Endpoint description contains all information that is necessary to establish a Secure Channel and a Session between the client and the server. The main parts are the network address of the server and the security settings like the server instance certificate, the security policy defining the used algorithms, and the type of user authentication used to create a Session.

Like FindServers this Service does not require a secure connection or a Session to be called. The parameters are described in Table 5.6.

Table 5.6 GetEndpoints Service parameters

Request parameters		Description
LocaleIds []		A list of locales for human readable strings returned in the endpoint descriptions. The server should use the first locale in the list that it supports
ProfileUris []		A list of profile URIs to allow the client to request Endpoints supporting specific transport profiles. All available Endpoints are returned if an empty list is passed in
Response parameters		Description
Endpoints []		A list of endpoint description structures for each returned Endpoint
	EndpointUrl	Network address of the Endpoint used by the client to establish a Secure Channel
	ServerCertificate	The server instance certificate used for the Endpoint. This is the public key of the server used by the client for securing the message exchange with the server
	SecurityPolicy	The security policy URI defining the algorithm sets and key length used for the Secure Channel. The security policy URIs are defined in [UA Part 7]
	SecurityMode	The message security mode used to secure the messages exchanged between client and server. Messages can be signed to detect changes of the message content and to ensure the right sender and messages can be encrypted to ensure privacy. The possible modes are SignAndEncrypt, Sign, and None
	UserIdentity Tokens []	A list of user identity tokens supported by the server to authenticate a user during the creation of a Session. Possible tokens are for example the combination of username and password, a certificate, or anonymous
	TransportProfileUri	URI of the network protocol used by the Endpoint. The transports profiles are defined in [UA Part 7]

5.3.3 Service RegisterServer

This Service registers a server with a discovery server. This Service will be called by a server or a separate configuration utility. Applications that are only client will not use this Service.

This Service requires a secured connection to make sure that only trusted servers can be registered. The registration call is done periodically by the server to indicate the availability. It is also called during shutdown to indicate that the server is shutting down and gets offline.

The Server passes all information to the Service that is necessary to return the application description in FindServers. In addition, the online status is sent to the Discovery Server.

5.4 Connection Management Between Clients and Servers

OPC UA requires establishing different levels of communication channels to ensure that all requirements for a secure, flexible, and reliable data communication are fulfilled (see Fig. 5.4).

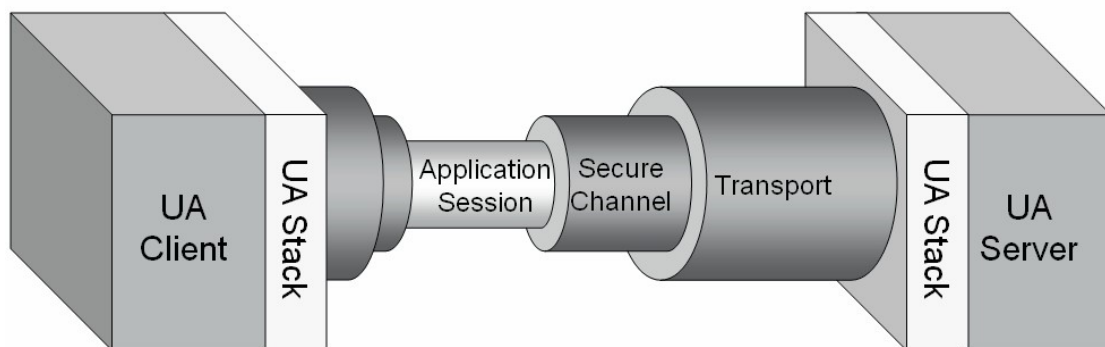


Fig. 5.4 Different levels of communication channels

The low - level network transport channel for exchanging the messages and the logical Secure Channel to secure the messages are handled by the UA Stacks. The Session on the application level used to authenticate users is handled by the client and server applications.

Most of the security handling is implemented by the UA Stacks provided by the OPC Foundation. On the application level, security handling is only necessary during handshaking for the Session establishment.

A client SDK will typically hide all of the security handling and combine the connection handling in connect and disconnect methods.

A server SDK implements all Session and connection handling without the need for a server implementer to do anything for the Secure Channel and Session Service Sets other than providing the configuration information.

5.4.1 Secure Channel Establishment

The SecureChannel Service Set is used to establish the low - level transport channel and the Secure Channel. Since most of the functionality is implemented in the UA Stacks, these Services are described from a Stack API point of view instead of the definition in the Service specification. These are the only two Services where the Stack APIs parameters are different than the parameters defined for the Services.

More details for the security related parts of the Secure Channel can be found in Chap. 7.

The Stack API parameters for OpenSecureChannel are described in Table 5.7. The Stack API parameters for CloseSecureChannel are described in Table 5.8. These Stack API parameters are different than the ones defined in [UA Part 4].

Table 5.7 Stack API parameters to open a Secure Channel

Stack API in parameters	Description
EndpointUrl	Network address of the server Endpoint used by the client to establish a Secure Channel
SecurityPolicy	The security policy URI defining the algorithm sets and key length used for the Secure Channel. The security policy URIs are defined in [UA Part 7]
SecurityMode	The message security mode used to secure the messages exchanged between client and server. Messages can be signed to detect changes of the message content and to ensure the right sender and messages can be encrypted to ensure privacy. The possible modes are SignAndEncrypt, Sign, and None
ServerCertificate	The server instance certificate used for the Endpoint. This is the public key of the server used by the client for securing the message exchange with the server
ClientCertificate	The client instance certificate. This is the public key of the client used by the server for securing the message exchange with the client
ClientPrivateKey	The private key for the client instance certificate. This private key is used by the client side UA stack to secure the message exchange with the server
RequestedLifetime	Requested lifetime for the security token. The security token must be renewed by the UA Stack before the lifetime expires. A renew starts after 75% of the lifetime has expired
Stack API Out parameters	Description
SecureChannelId	Identifier for the created Secure Channel
RevisedLifetime	Revised lifetime of the channel

Table 5.8 Stack API parameters to close a Secure Channel

Stack API In parameters	Description
SecureChannelId	Identifier for the Secure Channel to close
Stack API Out parameters	Description
No relevant out parameters	

5.4.2 *Creating an Application Session*

Like the Secure Channel Service Set, the Session Service Set is only used to establish a secure and reliable communication channel and not to transfer information from one system to another.

There are two Services for the handshaking to create a Session between two applications, one Service to close the Session and one Service to cancel Service calls in this Service Set.

The handshaking with the Services CreateSession and ActivateSession is necessary to ensure that the client can validate with CreateSession that the application he connects to is the server he wants to connect to and trusts in the further communication before sending sensitive data in ActivateSession like user name and password for the user authentication. The Service ActivateSession is also used to impersonate a user on the active Session and to assign another Secure Channel to the Session if the used Secure Channel is no longer valid.

5.4.2.1 **CreateSession Service**

This Service is used by an OPC UA client to create a Session and the server returns the identifier that is used in all the following Service invocations to assign them to the Session context.

A Session is not valid until the Service ActivateSession was called successfully. Sessions are terminated by the server if he did not receive Service calls during the negotiated timeout period. This ensures that the server can free resources in a defined way if the client is no longer available, for example based on a network interruption. This is a big advantage over Classic OPC specifications where the error handling is relying on fixed COM timeouts. Sessions should be closed gracefully by the client using the Service CloseSession if the communication channel is no longer needed.

A flag in the CloseSession Service indicates if all associated Subscriptions should be deleted. Each Subscription has its own lifetime and can be transferred to another Session if the Session is terminated or closed without deleting the associated Subscriptions.

There are several parameters in this Service only used for additional security validations and security handling. Examples are the EndpointDescription and the application certificates. They are already exchanged during Discovery or Secure Channel establishment but must be also validated in the context of a secure application Session. Since these parameters are typically handled by the UA SDKs and not visible to the application programmers, they are not listed here. See ref. [UA Part 4] for all parameters of this Service and Sect. 7.5.2 for more details on the security aspects of the secure communication channel (Table 5.9).

Table 5.9 CreateSession Service parameters

Request parameters	Description
SessionName	Name for the Session assigned by the client. The name is shown in the diagnostics information of the server
ClientDescription	Application description for the client application containing information like application and product URI or the application name
Requested SessionTimeout	Timeout of the Session requested by the client. If the client fails to issue a Service request within the interval negotiated with the server, the Session will automatically be terminate by the server
Response parameters	Description
SessionId	A unique public identifier assigned by the server to the Session. It is used to identify the Session in server diagnostic objects or in audit logs
AuthenticationToken	A unique private identifier assigned by the server to the Session. This identifier is only used for assigning Service calls to the Session and must be kept private between the client and the server application
Revised SessionTimeout	Timeout of the Session assigned by the server. This time typically matches the requested timeout of the client if it falls into the valid range defined by the server
ServerSoftware Certificates []	The list of software certificates of the server application identifying the product, the supported profiles, and the compliance test level for each profile

5.4.2.2 ActivateSession Service

The ActivateSession Service is used as the second part of the handshaking to establish an application Session. It is also used to change the user of the Session, to change the used language settings for a Session, and to assign a new Secure Channel to the Session (Table 5.10).

Table 5.10 ActivateSession Service parameters

Request parameters	Description
ClientSoftware Certificates []	The list of software certificates of the client application identifying the product, the supported profiles, and the compliance test level for each profile
LocaleIds []	List of locales that should be used by the server to provide localized strings. The server uses the first locale he supports in the list
UserIdentityToken	User identity token to validate and logon a specific user to the Session. There are different token types like user name and password or certificates supported. Section 7.5.3.4 describes the different token types
Response parameters	Description
Results []	Validation results for the client software certificates

5.4.3 Closing an Application Session

If a client does no longer need the connection to the server, he must use the Service CloseSession to start disconnecting from the Server and to free the Session resources in the server. The second step to disconnect is closing the Secure Channel. Both steps are typically combined in one disconnect method of a client SDK (Table 5.11).

Table 5.11 CloseSession Service parameters

Request parameters	Description
DeleteSubscriptions	A flag that indicates if the Server must delete all Subscriptions associated with the Session. Subscriptions can exist independent of a Session and can be transferred to another Session
Response parameters	Description

5.4.4 Cancel Outstanding Service Requests

A client is able to cancel outstanding Service requests by using the Service Cancel. To cancel requests can be helpful for potentially longer running Services like Query (Table 5.12).

Table 5.12 Cancel Service parameters

Request parameters	Description
RequestHandle	Request handle assigned by the client to one or more requests in the request header
Response parameters	Description
CancelCount	Number of canceled requests

5.5 Find Information in the Address Space

OPC UA provides capabilities to describe information and to transport this information. This section describes how to find different types of data in the Address Space of the server. The two main Services for this purpose are Browse to navigate through the Nodes in the Address Space and Read to access the metadata of the Nodes. Browse and the more specialized Services such as TranslateBrowsePathsToNodeIds, RegisterNodes, and UnregisterNodes are explained in this section. For a better understanding of how to use these generic Services, this section describes also the use of the Services and the Service parameters based on different typical use cases and types of information clients are interested in.

Since the Services are used to access the Information Model provided by the server, it is necessary to understand the concepts described in Chap. 2. For the simple use cases it is enough to know how Objects and Variables are connected together to provide data access capabilities. For the more enhanced use cases it is necessary to read Chaps. 2 – 4 completely.

5.5.1 Services Used for Discovering the Address Space

One of the main design goals of OPC UA was the combination of all types of Classic OPC information in one Address Space and the generic access to this model. Each of the Classic OPC specifications defined different ways to navigate through the Address Space and to access the available, but in most cases limited, type and metadata information. OPC DA, A&E, and HDA defined different but similar methods for browsing. DA and HDA defined completely different concepts of accessing properties of OPC variables and A&E defined eight methods to get information about the available event types. OPC UA covers all these different use cases with the flexible information modeling capabilities and the two Services Browse and Read.

5.5.1.1 Browse Service

The Browse Service is used by a client to navigate through the Address Space by passing a starting Node and browse filters and the server returns the list of Nodes connected to the starting Node by references.

The Browse Service takes a list of starting Nodes and returns a list of connected Nodes for each starting Node. Nevertheless, most clients will only pass one starting Node for the main purpose of building a tree hierarchy. Since the OPC UA Address Space can be a full-meshed network and is not limited to a pure hierarchy, the capability to pass in a list of starting Nodes is mainly used to browse metadata like the Properties of a list of Variables. A client SDK will provide different browse methods, one for a single starting Node and one for a list of starting Nodes.

Table 5.13 describes the parameters of the Browse Service.

Table 5.13 Browse Service parameters

Request parameters		Description
View		Passing a View allows limiting the browse to a specific View. For browsing the entire Address Space this parameter is not set
RequesteMax-ReferencesPerNode		Allows the client to limit the returned Nodes to protect against an unlimited number of results. BrowseNext can be used to get more results
NodesToBrowse []		Defines a list of starting Nodes and browse filters
	NodeId	NodeId of the starting Node
	BrowseDirection	Indicates if the server should follow references in the forward, the inverse, or both directions
	ReferenceTypeId	NodeId of the ReferenceType the server should follow. This parameter is typically combined with the IncludeSubtypes set to filter for a whole set of ReferenceTypes. This is for example Hierarchical-References to fill a tree control
	IncludeSubtypes	Indicates if also subtypes of the specified ReferenceTypeId should be returned by the server. Clients should set this value to true. Only in very seldom use cases it makes sense setting it to false
	NodeClassMask	Filter on the NodeClass of the returned Nodes, for example only requesting Objects and Variables
	ResultMask	Filter on the results returned per Node. The only information that is always returned is the NodeId of the target Node. All other result values can be excluded with this mask. This allows the client to reduce the server effort to find and return information the client is not interested in and reduces also the amount of data on the wire

(Continued)

Response parameters		Description
Results []		List of results for the passed starting Nodes and filters
	StatusCode	Result code for the passed starting Node and filter. This status code indicates only invalid filters or an unknown starting Node. An empty result list does not cause a failed status code
	ContinuationPoint	A continuation point is returned when the server was not able to deliver all results in the Browse response. The limitation can be set by the client in the request or by the server during Browse processing. The continuation point can be passed to BrowseNext to get the remaining results
	References []	
	Reference-TypeId	NodeId of the ReferenceType followed from the starting Node to the target Node
	IsForward	Indicates if the server followed a forward Reference or the inverse Reference from the starting Node to the target Node
	NodeId	NodeId of the target Node passing the filter criteria set in the request. This could be also a Node in another server indicated by the server information in the ExpandedNodeId
	BrowseName	The qualified name of the target Node. This name provides in some use cases a relation to the type system
	DisplayName	The localized name of the target Node used for display purposes. The locale depends on the Session setting defined in ActivateSession
	NodeClass	Indicates the NodeClass of the target Node
	TypeDefinition	NodeId of the Object or Variable type of the target Node. This parameter is only set if the target Node is a Variable or an Object

5.5.1.2 BrowseNext Service

This Service is only used to continue a Browse started with the Browse Service if not all results could be returned by the Browse or a following BrowseNext Service call. The number of Nodes to return can be limited by the client in the Browse request or by the Server during processing the Browse Service call. The parameters of the BrowseNext Service are described in Table 5.14.

Table 5.14 BrowseNext Service parameters

Request parameters	Description
ReleaseContinuation-Points	A flag that indicates if the Service is called only for releasing the memory associated with the continuation point in the server or if the next set of results should be returned Clients should always call this Service even if they do not want to continue browsing. In this case this flag is set to true
ContinuationPoints []	List of continuation points returned from a previous Browse or BrowseNext Service call
Response parameters	Description
This Service returns the same parameters like the Browse Service described in Table 5.13	

5.5.1.3 Read Service

The Browse Service returns already the Attribute values normally needed for filling up a browse tree like the display name, the NodeClass, or the TypeDefinition needed to display different icons for the different types of Nodes.

Additional Attributes needed for completing necessary information about Nodes like the data type or access level of Variables can be accessed using the Read Service. All available Attributes can be read by passing a list of NodeIds and AttributeIds as request parameters of the Read Service and the Server returns a list of Values for the requested NodeId and AttributeId combinations. The Read Service is described more detailed in Sect. 5.6.1.

5.5.1.4 TranslateBrowsePathsToNodeIds Service

This Service is used to access components of an Object based on the knowledge about the ObjectType. Since the NodeId of a Node is needed to access information provided by the Node like subscribing for Variable Value changes or to call a Method, it is necessary to know the NodeIds of components of Objects. Since OPC UA allows programming software components with built-in knowledge of ObjectTypes, UA needs to provide a mechanism to return the NodeIds for components of an Object instance based on the knowledge about the ObjectType. This mechanism is built on the requirement that the BrowseNames of components in the instance must be the same like BrowseNames of components in the type.

Figure 5.5 shows an example with the components of air conditioner controllers where the BrowseNames like Temperature or EngineeringUnit are defined by the ObjectType and the same BrowseNames are used on the two instances Controller1 and Controller2.

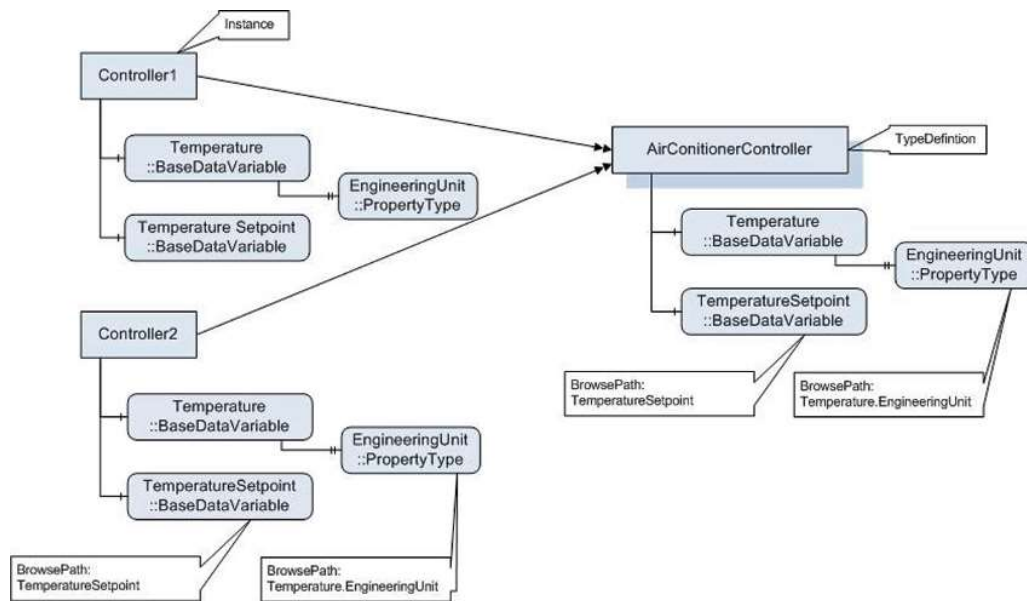


Fig. 5.5 Browse path in type definition and instance

Based on the requirement to have the same BrowseName, the browse path from the object type to a component is the same like the browse path from an instance to the same component.

For example a client wants to display the status of Controller1 in an air conditioner graphic. The graphic monitors the Value of the Variable Temperature and reads the Property EngineeringUnit to display the Value together with the unit. To get the NodeIds of the Variable and the Property, the graphic uses the Service TranslateBrowsePathsToNodeIds to pass in the starting Node Controller1 and the browse path “Temperature” for the Variable and the browse path “Temperature.EngineeringUnit” for the Property. The server returns then the NodeIds of the target Nodes. If more than one Node matches the browse path, the first Node in the list is the Node that is based on the type. The parameters of the Service are described more detailed in Table 5.15.

Table 5.15 TranslateBrowsePathsToNodeIds Service parameters

Request parameters		Description
BrowsePaths []		The list of browse paths for which NodeIds are requested
	StartingNode	NodeId of the Node where the server should start following the browse path
	RelativePath []	The browse path the server should follow. It is composed of a list of browse elements
	ReferenceTypeId	NodeId of the ReferenceType the server should follow. This parameter is typically combined with the IncludeSubtypes set to filter for a whole set of ReferenceTypes This is for example HasComponent to find the components of an Object

(Continued)

Response parameters		Description
	IncludeSubtypes	Indicates if also subtypes of the specified ReferenceTypeId should be followed by the server
	IsInverse	Indicates if the server should follow the ReferenceType in inverse direction. This flag is set to false for the default forward direction
	TargetName	BrowseName of the target Node. This name can be empty for the last element in the browse path. In this case all Nodes referenced by the specified ReferenceType are returned
Response parameters		Description
Results []		List of results for the passed starting Nodes and paths
	StatusCode	Result code for the passed starting Node and paths. If the path does not result in a target Node, BadNoMatch is returned by the server
	Targets []	List of target Nodes for each starting Node browse path combination. This list contains typically only one Node if the browse path is build with information from the type system
	TargetId	NodeId of the target Node
	RemainingPath-Index	Servers can have references to Nodes in other servers. In that case the full browse path cannot be processed by one server. Therefore the TargetId contains the starting Node in the other server and the client must pass the remaining path to this other server. This parameter defines the index of the starting element for the remaining path

5.5.1.5 RegisterNodes Service

This Service allows clients to optimize the cyclic access to Nodes for example for Writing Variable Values or for calling methods. There are two levels of optimization.

The first level is to reduce the amount of data on the wire for the addressing information. Since NodeIds are used for addressing in Nodes and NodeIds can be very long, a more optimized addressing method is desirable for cyclic use of Nodes. Classic OPC provided the concept to create handles for items by adding them to a group. RegisterNodes provides a similar concept to create handles for Nodes by returning a numeric NodeId that can be used in all functions accessing information from the server. The transport of numeric NodeIds is very efficient in the OPC UA binary protocol.

The second level of optimization is possible inside the server. Since the client is telling the server that he wants to use the Node more frequently by registering the Node, the server is able to prepare everything that is possible to optimize the access to the Node.

The handles returned by the server are only valid during the lifetime of the Session that was used to register the Nodes. Clients must call UnregisterNodes if the Node is no longer used to free the resources used in the server for the optimization. This method should not be used to optimize the cyclic read of data since OPC UA provides a much more optimized mechanism to subscribe for data changes. The parameters of the Service are listed in Table 5.16.

Clients do not have to use the Service and servers can simply implement the Service only returning the same list of NodeIds that was passed in if there is no need to optimize the access.

Table 5.16 RegisterNodes Service parameters

Request parameters	Description
NodesToRegister []	The list of Nodes to register. For each Node the NodeId of the Node is passed in
Response parameters	Description
RegisteredNodeIds []	List of NodeIds identifying the registered Node. This NodeId is typically an optimized numeric Node used as handle to the registered Node. This NodeId is only valid in the Session context it was created in. If the server does not know the NodeId or he is not able to optimize the access to the Node, he simply returns the NodeId provided in the request

UnregisterNodes Service

Handles created with the Service RegisterNodes must be freed by the client using the Service UnregisterNodes to free the resources in the server. The parameters of the Service are listed in Table 5.17.

Table 5.17 UnregisterNodes Service parameters

Request parameters	Description
NodesToUnregister []	The list of Nodes to unregister. For each Node the registered NodeId is passed in
Response parameters	Description

5.5.2 Use Cases for Finding Information in the Address Space

OPC UA provides different information in different levels of complexity in one generic and extensible model accessed by a small set of generic Services. For a better understanding about the use of the Services to find information necessary

for different use cases, this section describes the use of the Services for specific use cases. Some of the use cases are known from Classic OPC, other use cases are only possible with the new features provided by OPC UA.

5.5.2.1 Search Data Variables for Reading and Monitoring Data

The most common use case for OPC is to access Variable Values to read and write the current Value or to monitor the Value for data changes. In most cases, a user must select the list of Variables the client software uses for read, write, and monitoring data changes. This selection includes navigating through the Address Space to find the available Variables and to select the right Variables by checking the Attributes like data type and access level.

The Browse Service is used to navigate through the Address Space to find Variables. Table 5.18 describes the Browse parameters used to fill a browse tree.

Table 5.18 Browse parameters used to find Variables

Parameter	Value
View	Not set for browsing the whole Address Space, otherwise set to a view restricting the Address Space
RequestedMaxReferencesPerNode	500 is a good compromise between an efficient transport and a protection of the client. This should be adapted to the needs or restrictions of the tree display. If the server has more than 500 Nodes in one level, BrowseNext can be called with the returned continuation point
NodeId	Objects folder or one of the server Views as starting point for the browse. Following Browse requests use objects returned by previous Browse requests
BrowseDirection	Forward
ReferenceTypeId	HierarchicalReferences
IncludeSubtypes	True
NodeClassMask	Object and View – Building the hierarchy Variable – Variables providing Values
ResultMask	DisplayName – Used to display the name in the tree NodeClass – Used to distinguish between Objects and Variables TypeDefinition – Can be used to display different icons for different ObjectTypes or VariableTypes. For example to distinguish between Folder objects and other Objects or between Properties and DataVariables. ReferenceTypeId – Used to filter results for HasNotifier and HasEventSource since they are used to build an Event hierarchy and they are not relevant for data access

Only one starting Node is needed which allows using a simplified client SDK browse method that is reduced to one starting Node.

The information returned by the Browse can be used to identify a Node as Variable with the information NodeClass and TypeDefinition and provides the addressing information NodeId also returned by the Browse.

In addition a client needs other information like the data type of the Variable and the access level or the user access level to know if the Variable is readable or writable. The Read Service can be used to read this information for a list of Variables with one Read call. Table 5.19 provides a list of Attributes containing important metadata about a Variable. If a client wants to read these 5 Attributes for 10 Variables, he needs to call Read with a list of 50 elements containing a NodeId and AttributeId pair for each possible combination.

Table 5.19 Variable Attributes containing metadata

Attribute	Use
DataType	NodeId of the DataType. This could be one of the UA defined built-in types or a complex data type
ValueRank	Points out if the value is scalar, an array or a multidimensional array indicating the dimension
ArrayDimensions	An array of integers indicating the length of each dimension
AccessLevel	A bit mask indicating if the Value is readable or writable in the system. There are additional bits defined indicating the historical access capabilities
UserAccessLevel	The same bit mask used for AccessLevel but this Attribute can differ from the AccessLevel based on the user that is logged into the Session. For example is the AccessLevel for a set point typically readable and writable but the UserAccessLevel is only readable for a normal user that is not allowed to change a set point

In addition to reading the metadata of the Variables, a client may also be interested in Properties containing additional metadata for the Variables. To find such Properties, a Browse is called with a list of starting Nodes for each Variable that should be checked for availability of Properties. The same mechanism can be used to get the Properties of Objects or other Nodes (Table 5.20).

For the returned Property Nodes the Property Value can be determined by reading the Value Attribute of the Nodes with the Read Service.

Table 5.20 Browse parameters used to find Properties

Parameter	Value
NodesToBrowse []	List of Variables and filters
NodeId	NodeId of the Variable
BrowseDirection	Forward
ReferenceTypeId	HasProperty
IncludeSubtypes	True
NodeClassMask	Variable – Properties have the NodeClass Variable
ResultMask	DisplayName – Localized name of the Property BrowseName – Type name of the Property

5.5.2.2 Find Variables with Historic Data

The Variables providing historical data can be found in the same way like described in Sect. 5.5.2.1 for the current data. The only difference is that different flags must be checked for the AccessLevel and UserAccessLevel Attribute of the Variables. The additional flags are indicating if the history of data is available for reading and if the history can be updated.

There is another history - specific Attribute with the name Historizing. This flag indicates if the server is currently collecting history for the Value. Additional information can be found in the HistoricalConfiguration Object available for each Variable containing history by following the HasHistoricalConfiguration Reference from the Variable to the configuration Object.

5.5.2.3 Get Information to Call a Method

Methods are typically the target of a HasComponent Reference starting from an Object. Methods can be requested together with Variables and Objects when browsing an Object like described in Table 5.18. The NodeClass Method must be added to the NodeClassMask parameter settings described in this table. The parameters used to Browse only for Methods are described in Table 5.21.

Table 5.21 Browse parameters used to find Methods

Parameter	Value
NodeId	NodeId of the Object that provides Methods
BrowseDirection	Forward
ReferenceTypeId	HasComponent
IncludeSubtypes	True
NodeClassMask	Method – Filter for Methods
ResultMask	DisplayName – Used to display the name in the user interface BrowseName – Used to distinguish between known Methods from the ObjectType

The necessary information to call a Method is the `NodeId` of the Method and of the Object in which context the Method should be called. Both `NodeIds` are known after browsing for the available Methods. If the Method is not a well-known Method, the clients need to get also the description of the input and the output arguments for example to populate a user interface with this information.

There are two steps necessary to get this information, the first step is to find out the `NodeIds` of the Properties containing the argument descriptions and the second step is to read the values of the Properties to get the argument description. For the first step the most efficient way to get the `NodeIds` is to use the Service `TranslateBrowsePathsToNodeIds` since we know the `BrowsePath` from a Method to the Properties `InputArgument` and `OutputArgument` and the Service returns exactly the information we need. If we would use the `Browse` Service instead we may get additional Nodes we are not interested in. The necessary parameters for the `TranslateBrowsePathsToNodeIds` Service call are described in Table 5.22.

The client must check the result `StatusCode` for both `BrowsePaths` since the server will return the `BadNoMatch` `StatusCode` for a path if the Method does not have input or output arguments.

Table 5.22 `TranslateBrowsePathsToNodeIds` Service parameters

Request parameters		Description
BrowsePaths [0]		BrowsePath for Property <code>InputArguments</code>
	StartingNode	<code>NodeId</code> of the Method
	RelativePath [0]	Browse path with one element
	Reference-TypeId	HasProperty to find the Property of the Method
	IncludeSubtypes	True since the server may use a subtype of HasProperty
	IsInverse	False
	TargetName	BrowseName with the text <code>InputArguments</code> and the namespace index zero
BrowsePaths [1]		BrowsePath for Property <code>OutputArguments</code>
	StartingNode	<code>NodeId</code> of the Method
	RelativePath [0]	Browse path with one element
	Reference-TypeId	HasProperty to find the Property of the Method
	IncludeSubtypes	True since the server may use a subtype of HasProperty
	IsInverse	False
	TargetName	BrowseName with the text <code>OutputArguments</code> and the namespace index zero

For the returned NodeIds the client can call the Read Service to read the Value Attribute of the Properties. The Read returns an array of Argument structures for both Properties. The Argument structure describes a Method argument and contains the information argument name, description, and data type. The DataType of an argument can be complex. This allows using nested structures in Method arguments.

5.5.2.4 Find the Type Description for a Structured DataType

There are different use cases where a client needs to get the type description of a complex DataType but we assume that the NodeId of the type is already known from one of the following places:

- DataType Attribute of a Variable
- DataType parameter of an Argument structure describing the parameters of a Method
- DataType Attribute of an Event Field Variable.

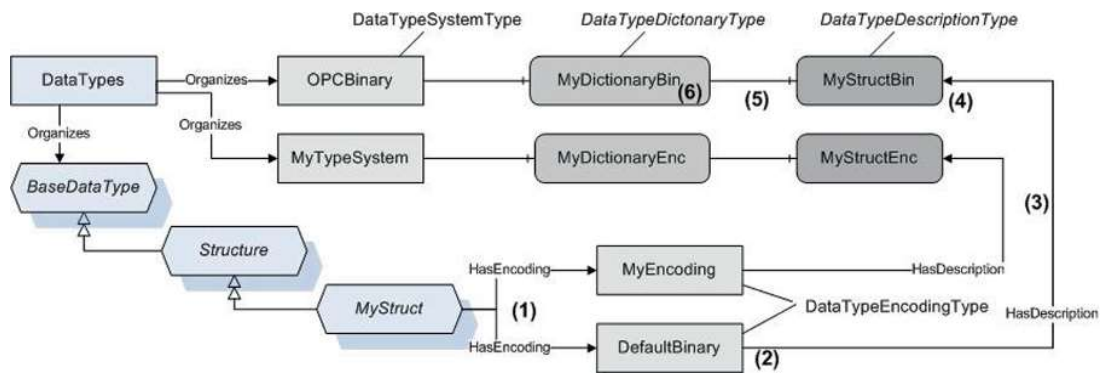


Fig. 5.6 Nodes involved to describe complex data types

Since OPC UA allows the support of different encodings for a complex DataType, the type description is not directly available through the DataType Node. Section 2.8.4 provides more details on structured DataTypes. The example used to explain the Service calls necessary to get the type description is shown in Fig. 5.6. The server in the sample has two DataTypeSystems, one is the OPCBinary and one is a vendor specific type system. The structured DataType MyStruct has the two corresponding encodings.

The DataType NodeId for MyStruct is used as starting point to find the type description with the following steps and Service calls:

1. To get the available data encodings for the DataType, the Browse Service is called to follow the ReferenceType HasEncoding from the DataType Node to the available DataTypeEncoding Objects.

2. The client must select an encoding from the list of returned `DataTypeEncoding` Objects. The selection can be done based on the preferred encoding or based on a user selection. In our example, the client is a generic client and therefore selects the `DefaultBinary` encoding.
3. From the `DefaultBinary` encoding the `DataTypeDescription` Variable can be browsed by following the `HasDescription` `ReferenceType`.
4. The client must read the Value Attribute of the `DataTypeDescription` Variable `MyStructBin` to get the identifier of the description used to find the entry in the `DataTypeDictionary`.
5. To find the `DataTypeDictionary` containing the description, the `Browse` Service needs to be called following the inverse `HasComponent` `ReferenceType` from the `DataTypeDescription` Node to the `DataTypeDictionary`:
6. The client must first check if the returned `DataTypeDictionary` `MyDictionaryBin` is already in the client cache to avoid multiple reads of potentially large dictionaries. If the dictionary is not available, the client can read the dictionary by calling `Read` for the Value Attribute of the Variable `MyDictionaryBin`. The description can be found in the dictionary with the identifier read from the `MyStructBin` Variable.

5.5.2.5 Find Object Components Based on ObjectType Knowledge

If a client software component was built with the knowledge about an `ObjectType`, the component knows the path from the type to `InstanceDeclarations`. This knowledge is necessary to find the used components of an Object instance based on this knowledge.

If the Object has for example three Variables the software component needs to monitor, the `NodeIds` of these three Variables are needed. To get the `NodeIds` the client calls the `TranslateBrowsePathsToNodeIds` Service with three browse paths containing the Object instance `NodeId` as starting Node and the relative path from the `ObjectType` to the `InstanceDeclaration` of the Variable.

5.5.2.6 Search Event Hierarchy and Fill up an Event Filter Display

The rules to create a filter are defined by OPC UA and are described in the Sect. 5.7.5 for Event monitoring. The server - specific parts of the Event filters are Event Types and the Event fields defined by the EventTypes and sources for Events which can be found in the Object and Variable instances Nodes.

Most of the information necessary to populate an Event filter dialog can be found in the EventType hierarchy. There is a well-known `NodeId` for the `BaseEventType` which is the root Node for the EventType hierarchy. Table 5.23 describes the Browse parameters used to fill a tree control with the EventType hierarchy.

The Event fields from the tree are used to select the fields delivered with an Event notification and are used together with the EventTypes to filter Events.

Table 5.23 Browse parameters used to find EventTypes and the Event Fields

Parameter	Value
NodeId	BaseEventType ObjectType as starting point for the browse Following Browse requests use derived EventType Nodes returned by previous Browse requests
BrowseDirection	Forward
ReferenceTypeId	HierarchicalReferences
IncludeSubtypes	True
NodeClassMask	ObjectType – Building the Type hierarchy Object – Potentially used to structure Event Types and Event Fields in the Address Space. Only InstanceDeclarations, i.e. Objects referencing a ModellingRule with the HasModellingRule, should be considered Variable – Variables representing the fields of an Event Type. These Variables are typically Properties. Only InstanceDeclarations should be considered
ResultMask	DisplayName – Used to display the name NodeClass – Used to distinguish between ObjectTypes, ³ Objects and Variables TypeDefinition – Can be used to display different icons for different Object or Variable types. For example to distinguish between Folder objects and other Objects or between Properties and DataVariables

A second tree control can be populated with EventNotifiers⁴ and Event sources provided by the server. The References used to build such a hierarchy are Organizes, HasNotifier, and HasEventSource. Starting points for such hierarchies are the Server Object, the Objects Folder, or Views. Table 5.24 describes the Browse parameters used to fill a tree control with the EventNotifier and Event source hierarchy.

The EventNotifiers are used to create Monitored Items which is a preselection of Events provided by this EventNotifier. The Event sources can be used as filter criteria.

³ObjectTypes represent Event Types in this case.

⁴An EventNotifier is an Object where the SubscribeForEvents flag is set in the EventNotifier Attribute.

Table 5.24 Browse parameters used to find EventNotifiers

Parameter	Value
NodeId	Server Object, the Objects Folder or Views as starting point for the browse Following Browse requests use Objects returned by previous Browse requests
BrowseDirection	Forward
ReferenceTypeId	HierarchicalReferences
IncludeSubtypes	True
NodeClassMask	Object – Building the hierarchy Variable – Variables can be Event sources
ResultMask	DisplayName – Used to display the name NodeClass – Used to distinguish between Objects and Variables TypeDefinition – Can be used to display different icons for different Object or Variable types. For example to distinguish between Folder objects and other Objects or between Properties and Data-Variables ReferenceTypeId – Used to filter results for Organizes, HasNotifier and HasEventSource. All other references are not relevant to the Event hierarchy

5.5.2.7 Find Information for a State Machine Display

State Machines are one of the base concepts of OPC UA. The base concept can be used by servers to describe application specific State Machines but it is also used by OPC Information Models like Programs [UA Part 10] and Alarm & Conditions [UA Part 9]. The base concepts and OPC Information Models using these concepts are described in Chap. 4.

This section describes the discovery of a State Machine description based on an abstract example but the same concepts can also be used to discover also Program or Alarm State Machines. Since it is expected that clients understanding the OPC UA Program or Alarms and Conditions Information Model do not need to browse the well-known State Machines, this is only necessary for vendor – specific extensions to these Information Models.

The abstract example used to explain the concepts of browsing a State Machine is shown in Fig. 5.7. The ReadingUaBook State Machine and the corresponding State Machine Type has two states, Idle and Reading and the transitions between the two states.

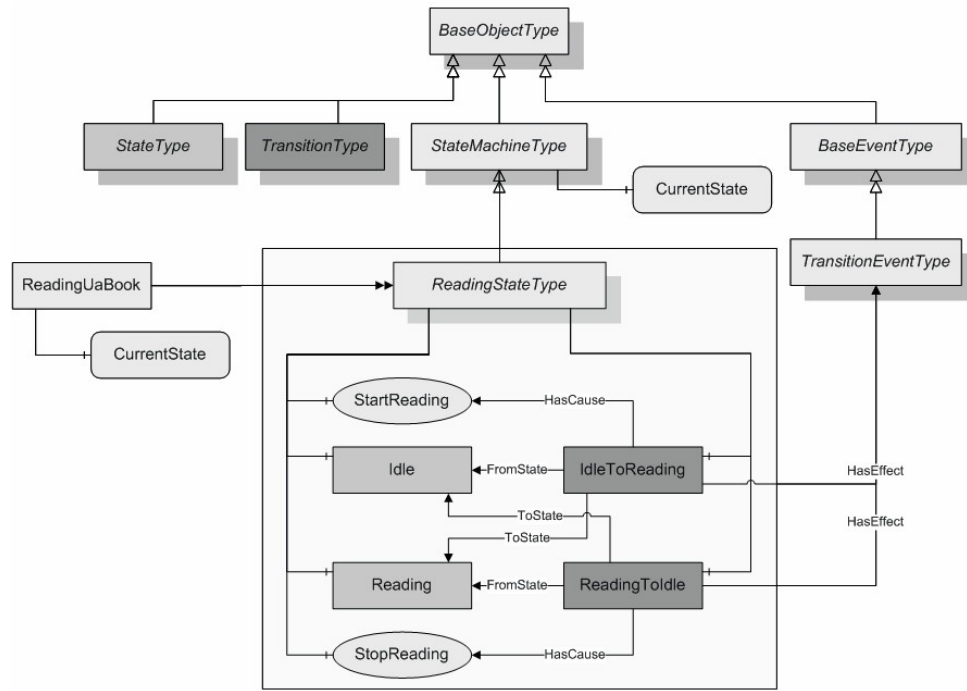


Fig. 5.7 ReadingUaBook State Machine and its Type Definition

If a client finds a StateMachine instance like the ReadUaBook Object in a server, the information provided by the instance itself is limited to information related to the current state. All information describing the State Machine like possible states and transitions is only available on the StateMachineType.

Two NodeIds are needed as starting points. The NodeId of the State Machine instance ReadingUaBook is necessary to get information about the current state and to subscribe for transition Events. The NodeId of the StateMachineType ReadingStateType is necessary to get the description of the State Machine. The NodeId of the TypeDefinition ReadingStateType is normally known from browsing the instance or can be browsed by following the HasTypeDefinition Reference from the instance ReadingUaBook.

Using the ReadingStateType NodeId as starting point the State Machine description can be discovered with the following steps and Service calls:

1. To get the available states, transitions, Methods, and Variables containing additional information, the Browse Service is called with the following settings:
 - ReadingStateType NodeId as starting Node
 - BrowseDirection set to Forward
 - ReferenceTypeId set to HasComponent
 - IncludeSubtypes set to True
 - NodeClassMask set to Object, Method, and Variable
 - and Result Mask set to DisplayName, BrowseName, and TypeDefinition

2. To find the relations between the returned components, the client takes the list of NodeIds of the transition Nodes as list of starting Nodes to four Browse calls with the following settings:
 - IdleToReading and ReadingToIdle Object NodeIds as starting Node
 - BrowseDirection set to Forward
 - ReferenceTypeId set to HasCause, HasEffect, FromState, and ToState in the four different Browse calls
 - IncludeSubtypes set to True
 - NodeClassMask set to all
 - and Result Mask set to DisplayName and BrowseName

Based on the information returned from the Browse calls the client can for example display the State Machine with the states and transitions, he can provide a way to trigger state changes with the available Methods and by reading the current state from the instance and subscribing for the transition Events, the client can monitor the state of the State Machine.

5.6 Read and Write Data and Metadata

One of the most important features of OPC is to read and write data from another system using a standardized data exchange mechanism.

The Read and Write Services not only allow reading and writing the Values of Variables, but are also used in a generic way to read and write Attributes of Nodes to access metadata in the Address Space. A different way to read data is the subscription for data changes. This is the preferred method for clients needing cyclic updates of variable value changes.

5.6.1 Reading Data

The Read Service is used to read one or more Attributes of one or more Nodes. It allows also reading subsets or single elements of array values and to define a valid age of values to be returned to reduce the need for device reads. Like most other Services, the Read Service is optimized for bulk read operations and not for reading single Attribute values. Typically all Node Attributes are readable. For the Value Attribute the Read rights are indicated by the AccessLevel and UserAccessLevel Attribute of the Variable. The parameters of the Read Service are described in Table 5.25.

Table 5.25 Read Service parameters

Request parameters		Description
MaxAge		The maximum age of the value to be read in milliseconds. This parameter allows clients to reduce the communication between server and data source by allowing the server to return a cached value that is not older than the defined time period. Setting a value of 0 forces the server to obtain the current value. This is similar to a device read in Classic OPC
TimestampsToReturn		OPC UA defines two timestamps, the source and the server timestamp. This parameter allows the client to define which timestamps the server should return with the value. See the response parameters for a description of the different timestamps
NodesToRead []		List of Nodes and Attributes to read
	NodeId	Identifier for the Node to read
	AttributeId	Identifier for the Attribute of the Node to read. This could be the Value Attribute or any other valid Attribute providing metadata for Nodes. A list of Attributes can be found in Appendix B
	IndexRange	This parameter is used to identify a single element of an array or a single range of indexes for arrays
	DataEncoding	This parameter is only relevant for reading values with a structured DataType. Structured types can be transported using different data encodings. Default encodings for UA are XML or UA binary format. This parameter allows the client to define the encoding used to transport the complex value
Response parameters		Description
Results []		List of read results contained in DataValue structures
	Value	Contains the read value if the StatusCode parameter indicates a successful read
	StatusCode	Success code for the read operation or quality of the read value. The value is only usable if the status is good
	SourceTimestamp	Source timestamp assigned to the value if requested by the client. The source timestamp is only available for Value Attributes The source timestamp is used to reflect the timestamp that was applied to a Variable value by the data source. It should indicate the last change of the value or status code. The source timestamp must be always generated by the same physical clock. This timestamp type was added for OPC UA to cover

(Continued)

		the use case to get the timestamp of the last value change which is different than the ServerTimestamp definition
	ServerTimestamp	Server timestamp assigned to the value if requested by the client The server timestamp is used to reflect the time that the server received a Variable value or knew it to be accurate if the changes are reported by exception and the connection to the data source is operating. This is the behavior expected by Classic OPC

5.6.2 Writing Data

The Write Service is used to write one or more Attributes of one or more Nodes. It allows also writing of subsets or single elements of array values. Like most other UA Services, the Write Service is optimized for bulk write operations and not for writing single Attribute values. The parameters of the Write Service are described in Table 5.26.

Table 5.26 Write Service Parameters

Request parameters		Description
NodesToWrite []		List of Nodes, Attributes, and values to write
	NodeId	Identifier for the Node to write
	AttributeId	Identifier for the Attribute of the Node to write. This could be the value Attribute or any other valid Attribute. A list of Attributes can be found in Appendix B
	IndexRange	This parameter is used to identify a single element of an array or a single range of indexes for arrays
	Value	Contains the value to write
	StatusCode	Status code assigned to the value. A zero value indicates that the status is not set
	SourceTimestamp	Source timestamp assigned to the value. A null value indicates that the timestamp is not set
	ServerTimestamp	Server timestamp assigned to the value. A null value indicates that the timestamp is not set
Response parameters		Description
Results []		List of write result status codes for each write operation

Typically only the Value Attributes of Variables are writable. Other Attributes are only writable if the server allows the configuration of Nodes through OPC UA. For the Value Attribute the Write rights are indicated by the AccessLevel or UserAccessLevel Attribute of the Variable. The Write rights for all other Attributes are indicated by the WriteMask or UserWriteMask Attributes.

This Service allows also writing the status and the timestamps of the value if it is supported by the server. There is a defined error status code returned if the client tries to write these parameters but the server does not support this feature.

5.7 Subscribe for Data Changes and Events

A client can subscribe for three different types of information from an OPC UA server. A Subscription is used to group sources of information together. A Monitored Item is used to manage a source of information. A piece of information is called a notification. A Subscription can contain all three different types of Monitored Items and the server will deliver notifications for these Monitored Items until the Subscription or the Monitored Items are deleted.

The first and most common type of Monitored Item is used to subscribe for data changes of Variable Values (Sect. 5.7.4). The second type of Monitored Item is used to subscribe for Events by defining an EventNotifier⁵ to monitor and by defining a filter for the Events (Sect. 5.7.5). The third type of Monitored Item is used to subscribe for aggregated Values calculated based on current Variable Values in client-defined time intervals (Sect. 5.7.6).

Most of the Services are used to create the necessary context for the Subscription (Sect. 5.7.2) and Monitored Items (Sect. 5.7.3). Figure. 5.8 shows the relation between the communication context created on top of a Session for subscribing to data changes or Events.

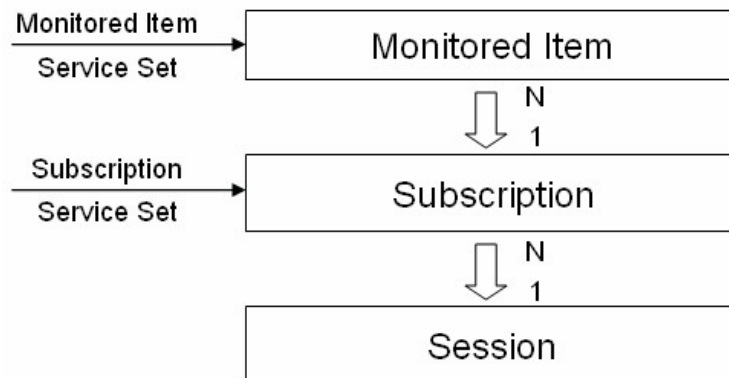


Fig. 5.8 Context necessary to subscribe for data changes and Events

All Monitored Items have common settings like monitoring mode, sampling interval, filter settings, and queue size. The different types of Monitored Items are defined by the type of source assigned to the item and the filter defined for the Monitored Item. Figure. 5.9 shows the different Subscription and Monitored Item settings.

⁵An EventNotifier is an Object where the SubscribeForEvents flag is set in the EventNotifier Attribute.

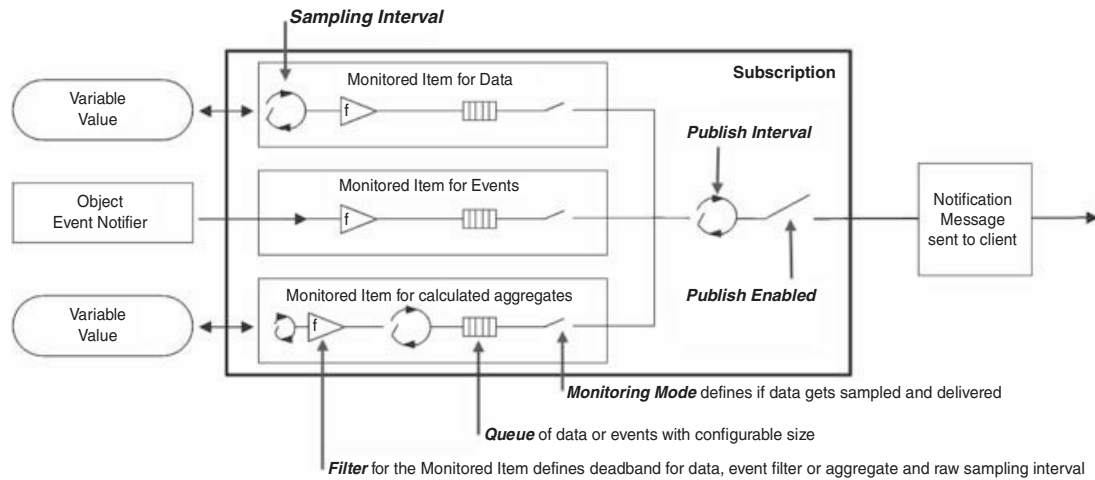


Fig. 5.9 Settings for Subscription and Monitored Items

The sampling interval defines the rate the server checks Variable Values for changes or defines the time the aggregate gets calculated. The monitoring mode defines if the Monitored Item is active or inactive. The queue size defines how many notifications can be queued for delivery. The default value for data changes is one and the value for Events is infinite where the size of infinite depends on the resources available in the server. The filter settings are different for data changes, Events, and aggregate calculation.

There are two Subscription settings. The Publish interval defines the interval when the server clears the queues and delivers the notifications to the client. The Publish enabled setting defines whether the data gets delivered to the client.

The only two Services used to actually deliver the notifications in a notification message to the client are the Publish Service for transferring the notification messages and the Republish Service to get lost notification messages from the server. The mechanism to deliver the information is described in Sect. 5.7.1. The Services used to create the necessary Subscription and MonitoredItem context are explained afterward.

5.7.1 Delivery of Changed Data and Events

The nature of the notifications provided by the Subscription requires a report by exception from the server to the client. It is required to combine notifications to larger notification messages to optimize interprocess or network communication which is the typical use case of OPC. Nevertheless the server needs to be able to send a notification message to the client whenever it is required.

In Classic OPC this was achieved by defining callback interfaces allowing the server to call methods on the client to send data change or event notifications. Based on the requirement to be firewall-friendly and to ensure the same behavior for all transports, OPC UA does not define such a callback interface. An early

version of the Services specified a callback channel but the use of the callback channel was reduced to the UA TCP protocol or required a bidirectional connection establishment for Web Services which is not possible through firewalls. Since the final mechanism to exchange notification messages through a one - way connection had no limitations, the callback channel approach was removed from the Service specification. The current mechanism is even more efficient and reliable taking all requirements for a communication between distributed systems into account.

The mechanism for a secure and reliable exchange of notification messages has the following requirements:

- Server-triggered sending of notification messages
- Sending a life ping from the client to the server
- Sending a life ping from the server to the client when no notification messages are available
- Sending sequence numbers together with the notification message from the server to the client to allow the client detecting lost messages
- Acknowledgement of received sequence numbers from the client to the server
- Resending of lost notification messages.

The Services used to implement this mechanism are the Publish and the Republish Services. The Services fulfill all these requirements based on the Service parameters and the special behavior defined for these Services. The parameters of the Services are described in the following sections after describing the base mechanism realized with Publish and Republish.

5.7.1.1 Server-Triggered Sending of Notification Messages

The server-triggered sending of notification messages is accomplished by a special rule for the Publish Service. It is the only Service that can be blocked by the Server without doing any processing. It is expected that a client sends a list of Publish requests to the server without expecting an immediate response. The server can queue the Publish requests until a notification message is ready for sending to the client. Since the exchange of request and response messages is asynchronous by definition, the communication is not blocked by these outstanding Publish requests. If the client uses the right algorithm for sending Publish requests, the server is able to trigger the sending of notification messages.

The algorithm for sending Publish requests from the client to the server depends on the following parameters:

- Number of Subscriptions
- Network latency
- Maximum queue size for Publish requests in the server.

The Publish request is not bound to a specific Subscription and can be used by the server for all Subscriptions running in the same Session context. To make sure that all Subscriptions can send a notification message at the same time, the client should make sure that there are more outstanding Publish requests than active Subscriptions.

Additional Publish requests may be required if the latency of the network connection is very high. This can be calculated based on the timestamps contained in the request and response messages. In the case of a large number of Subscriptions and low network latency, the number of outstanding Publish requests can be reduced.

If the Server indicates an overflow of his Publish queue with a Service result of `BadTooManyPublishRequests`, the client must reduce the calculated number of outstanding Publish requests.

Figure 5.10 summarizes the mechanisms used for the delivery of notification messages.

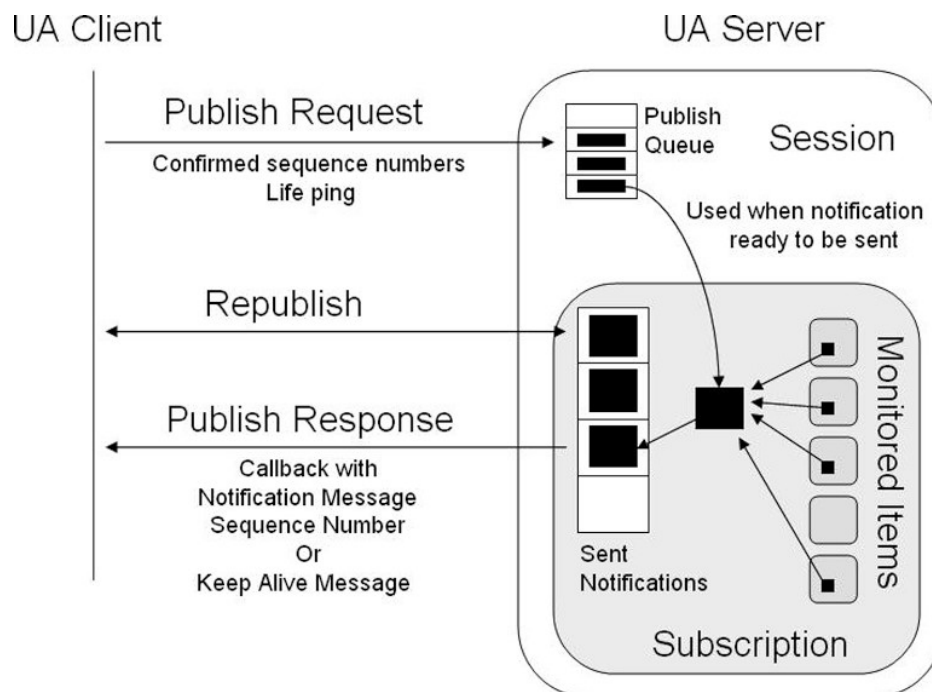


Fig. 5.10 Delivery of notification messages

5.7.1.2 Keep Alive Messages

An important requirement for a quick recovery from error scenarios is a timely detection of communication problems. One important feature is the configurable timeout for Service invocations in OPC UA. But this feature cannot be used to detect connection problems for the Publish since Publish requests are queued in the server and need therefore a much longer timeout than required for a timely detection of communication problems.

For this reason the Publish procedure provides life ping mechanisms in both directions. Every Publish request is a life ping from the client to the server. Every notification message is a life ping from the server to the client. If no notification message is available, an empty Publish response is sent to the client as life ping after the keep-alive interval. This interval is configurable by the client and is a multiple of the publishing interval used to send notification messages.

5.7.1.3 Detection and Resending of Lost Notification Messages

Another important requirement for a reliable communication is the detection of lost data. This can be achieved with the exchange of sequence numbers with each notification message. If a client detects missing sequences, he can use the Republish Service to get the lost notification message from the server. This is important for clients interested in all Value changes or for clients subscribing for Events.

Resending lost notification messages is also important for clients which are only interested in the latest values since the lost message may contain changes for Variables Values that have not be changed in the next notification message.

If a client received a notification message, he needs to acknowledge the sequence number in the next Publish request to allow the server to free the memory allocated for the buffered notification messages.

5.7.1.4 Real Callbacks and Other Help from SDKs

The Publishing mechanism is very powerful and has a certain complexity to fulfill all requirements for a secure, reliable, and high performance communication through firewalls. But the mechanisms are only necessary for a remote communication and can therefore be hid by a client-side UA SDK providing a real callback inside the client application. All complexity can be completely hidden by the SDK.

On the server-side a SDK can do even more since the whole Subscription handling including the Publish Service handling is typically implemented by a server-side UA SDK. Only the sampling of the data or the event monitoring needs to be done by the server application.

5.7.1.5 Publish Service

The parameters of the Publish Service are described in Table 5.27. The relevant Subscription settings for the trigger of the Publish response like the publishing interval or the keep-alive interval are negotiated during the creation of the Subscription.

Table 5.27 Publish Service parameters

Request parameters		Description
Subscription Acknowledgements []		List of sequence numbers the client received and where the server can free resources for
	SubscriptionId	The id of the Subscription that sent the notification message
	SequenceNumber	Sequence number of the received notification message to acknowledge
Response parameters		Description
SubscriptionId		The id of the Subscription sending the notification message
Available SequenceNumbers []		A list of sequence numbers available in the Subscription for retransmission and not acknowledged by the client
MoreNotifications		A flag that indicates if the server was not able to send all available notifications in this Publish response
NotificationMessage		Structure containing the notification message
	SequenceNumber	Sequence number of the notification message
	PublishTime	The time that this message was sent to the client
	NotificationData []	A list of extensible parameters containing the notification data. This could be a DataChange notification or an Event notification. Since only two notification data types are defined yet, this list can have a size of one or two elements

5.7.1.6 Republish Service

The parameters of the Republish Service are described in Table 5.28.

Table 5.28 Republish Service Parameters

Request parameters		Description
SubscriptionId		The id of the Subscription that sent the notification message
Retransmit SequenceNumber		Sequence number of the notification message to resend
Response parameters		Description
NotificationMessage		Structure containing the notification message
	SequenceNumber	Sequence number of the notification message
	PublishTime	The time that this message was sent to the client
	NotificationData []	A list of extensible parameters containing the notification data. This could be a DataChange notification or an Event notification. Since only two notification data types are defined yet, this list can have a size of one or two elements

5.7.2 Create and Manage Subscriptions

In normal operation the relevant settings for the Subscription are the Publish enabled and the Publish interval. Additional settings are keep-alive count, lifetime count, maximum number of notifications per Publish, and the priority of the Subscription.

The keep-alive count defines how many times the Publish interval needs to expire without having notifications available before the server sends an empty message to the client indicating that the server is still alive but no notifications are available.

The lifetime count defines how many times the Publish interval expires without having a connection to the client to deliver data. If the server is not able to deliver notification messages after this time, it deletes the Subscription to clear the resources. The lifetime count must be at minimum three times the keep-alive count. Both values are negotiated between the client and the server.

The maximum number of notifications per Publish is used to limit the size of the notification message sent from the server to the client. The number of notifications is set by the client but the server can send fewer notifications in one message if his limit is smaller than the client-side limit. If not all available notifications can be sent with one notification message, another notification message is sent.

The priority setting defines the priority of the Subscription relative to the other Subscriptions created by the Client. This allows the server to handle Subscriptions with higher priorities first in high-load scenarios.

5.7.2.1 CreateSubscription Service

This Service is used to create a Subscription and to define the initial settings for the Subscription. The Subscription can be deleted using the DeleteSubscriptions Service or by setting the DeleteSubscriptions flag when closing the Session. The parameters of the CreateSubscription Service are described in Table 5.29.

Table 5.29 CreateSubscription Service parameters

Request parameters	Description
Requested PublishingInterval	Client-requested publishing interval for the Subscription
PublishingEnabled	Publish enabled setting for the Subscription
Requested MaxKeepAliveCount	Client-requested keep-alive count for the Subscription
Requested LifetimeCount	Client-requested lifetime count for the Subscription
MaxNotifications PerPublish	Client-defined maximum number of notifications per notification message delivered to the client by Publish

(Continued)

Priority	Priority of the Subscription in the client relative to other Subscriptions created by the client
Response parameters	Description
SubscriptionId	Id of the subscription set by the server. The client must use this id in all following Service calls related to this Subscription
Revised PublishingInterval	Server-revised publishing interval for the Subscription
Revised MaxKeepAliveCount	Server-revised keep-alive count for the Subscription
RevisedLifetimeCount	Server-revised lifetime count for the Subscription

5.7.2.2 DeleteSubscriptions Service

This Service is used to delete a list of Subscriptions created with the CreateSubscription Service. The parameters of the DeleteSubscription Service are described in Table 5.30.

Table 5.30 DeleteSubscriptions Service parameters

Request parameters	Description
SubscriptionIds []	List of Subscriptions to delete. The Subscriptions are identified by their ids created in the CreateSubscription Service
Response parameters	Description
Results []	List of status codes for the passed Subscription ids indicating if the delete was successful

5.7.2.3 ModifySubscription Service

This Service is used to modify the settings of a Subscription. The parameters of the ModifySubscription Service are described in Table 5.31.

Table 5.31 ModifySubscription Service parameters

Request parameters	Description
SubscriptionId	Id of the Subscription to modify. This id was returned from the CreateSubscription Service
Requested PublishingInterval	Client-requested publishing interval for the Subscription
Requested MaxKeepAliveCount	Client-requested keep-alive count for the Subscription

(Continued)

RequestedLifetimeCount	Client-requested lifetime count for the Subscription
MaxNotificationsPerPublish	Client-defined maximum number of notifications per notification message delivered to the client by Publish
Priority	Priority of the Subscription in the client relative to other Subscriptions created by the client
Response parameters	Description
RevisedPublishingInterval	Server-revised publishing interval for the Subscription
RevisedMaxKeepAliveCount	Server-revised keep-alive count for the Subscription
RevisedLifetimeCount	Server-revised lifetime count for the Subscription

5.7.2.4 SetPublishingMode Service

This Service is used to modify the Publish-enabled setting for a list of Subscriptions. The parameters of the SetPublishingMode Service are described in Table 5.32.

Table 5.32 SetPublishingMode Service parameters

Request parameters	Description
PublishingEnabled	A flag that indicates if the Publish should be enabled or disabled for the list of Subscriptions passed to this Service
SubscriptionIds []	List of Subscriptions to modify. The Subscriptions are identified by their ids returned by the CreateSubscription Service
Response parameters	Description
Results []	List of status codes for the passed Subscription ids indicating if the setting of the Publish-enabled flag was successful

The Service allows clients to deactivate the delivery of notification messages without deactivating the collection of data and events.

5.7.2.5 TransferSubscriptions Service

This Service is used to transfer a list of Subscriptions to the Session that is used to call this Service. This feature is used by redundant clients to transfer a Subscription from the main client to the backup client if the main client is no longer available. It is also used to assign a Subscription to a new Session if the old one is no longer valid but the Subscription is still valid. The parameters of the TransferSubscriptions Service are described in Table 5.33.

Table 5.33 TransferSubscriptions Service parameters

Request parameters		Description
SubscriptionIds []		List of Subscriptions to transfer. The Subscriptions are identified by their ids returned by the CreateSubscription Service
Response parameters		Description
Results []		List of transfer results
	StatusCode	Result of the transfer operation for one Subscription
	AvailableSequenceNumbers []	A list of sequence numbers available in the Subscription for retransmission and not acknowledged by the client

5.7.3 Create and Manage Monitored Items

Clients are creating Monitored Items in a Subscription to subscribe for data changes and Events. This section describes the common settings for Monitored Items and the Services to manage Monitored Items. The three different types of Monitored Items are described in more detail in the following sections.

The sampling interval of the Monitored Item defines the rate in milliseconds; the underlying data source is sampled for data changes. The sampling interval can be inherited from the publish interval of the Subscription but can be also set individually for each item. If the server must sample the data source, there is typically a minimum possible rate. If this minimum is known, it is exposed as MinimumSamplingInterval Attribute of the Variable. If the data source delivers the data exception based at changes, the server can also accept sampling intervals of 0. The server can adjust the requested rate to the next possible rate it supports but the server must attempt to sample at the defined rate, however, the server is not allowed to sample faster than the negotiated rate. The sampling interval is 0 for Event Monitored Items.

The monitoring mode defines the states disabled, sampling, and reporting. The difference between sampling and reporting is that for sampling the data source is sampled but the notifications are not sent to the client and for reporting they are also sent to the client. The setting sampling is necessary for the definition of triggered monitored items where the notifications are only reported when a triggering Monitored Item has a new notification. More details on triggering are described for the SetTriggering Service in this section.

Filter settings are specific to the different types of Monitored Items and are described in the following sections. The filter defines if a notification gets queued for transfer to the client.

The queue parameters define the size of the queue for notification message and the discard policy if the queue is full but a new notification message is available and passed the filter criteria. In this case it could be defined that the newest or the

oldest notification in the queue will be overwritten. The queue size for Event Monitored Items is unlimited. For data Monitored Items it depends on the use case. A HMI client only displaying the latest value will set the size to one. A size greater than one can be used by clients that do not want to lose data changes even if the sampling is faster than the delivery of the notification message with Publish. This applies for example for a trend display.

5.7.3.1 CreateMonitoredItems Service

This Service is used to create Monitored Items in a Subscription and to define the initial settings for the Monitored Items. They can be deleted using the DeleteMonitoredItems Service or by deleting the Subscription. The parameters of the CreateMonitoredItems Service are described in Table 5.34.

Table 5.34 CreateMonitoredItems Service parameters

Request parameters		Description
SubscriptionId		Id of the Subscription to add the items to. This id was returned from the CreateSubscription Service
TimestampsToReturn		OPC UA defines two timestamps, the source and the server timestamp. This parameter allows the client to define which timestamps the server should return with the value. The two different timestamps are described in Sect. 5.6.1
ItemsToCreate []		List of items to create with the requested settings
	NodeId	NodeId of the Node to monitor. This is typically a Variable for data and must be an Object for Events. This can be also all other Node classes for data monitoring but this would be an unusual use case
	AttributeId	Id of the Attribute to monitor. This could be the Value Attribute of a Variable for data or the Event-Notifier Attribute for Events It is possible to subscribe for data changes of all defined Attributes for all possible Node classes. But there are more efficient methods (see Sect. 5.7.4) to get informed about changes of Attributes other than Value since it is expected that these Attributes change never or only after configuration changes
	MonitoringMode	Monitoring mode of the item which could be disabled, sampling, or reporting
	ClientHandle	Client defined handle for the Monitored Item. This handle gets delivered together with the notification to allow the client to assign the notification to a Monitored Item

(Continued)

	Monitoring Parameters	Client requested monitoring parameters sampling interval, filter, queue size, and discard policy
Response parameters		Description
Results []		List of results for the list of items to create
	Status Code	Status code that indicates if the creation of the requested Monitored Item succeeded
	MonitoredItemId	Id for the Monitored Item assigned by the server This id must be passed in Services used to modify and delete Monitored Items
	Revised SamplingInterval	Sampling interval accepted by the server. If the requested rate is not available, the server adjusted the interval to the next available longer rate
	RevisedQueueSize	Queue size used by the server

5.7.3.2 DeleteMonitoredItems Service

This Service is used to delete MonitoredItems created with CreateMonitoredItems. The parameters of the DeleteMonitoredItems Service are described in Table 5.35.

Table 5.35 DeleteMonitoredItems Service parameters

Request parameters		Description
SubscriptionId		Id of the Subscription to remove the items from. This id was returned from the CreateSubscription Service
MonitoredItemIds []		List of Monitored Items to delete. The ids were returned from the CreateMonitoredItems Service
Response parameters		Description
Results []		List of status codes for each item to delete indicating if the delete succeeded

5.7.3.3 ModifyMonitoredItems Service

This Service is used to modify MonitoredItems in the Subscription. The parameters of the ModifyMonitoredItems Service are described in Table 5.36.

Table 5.36 ModifyMonitoredItems Service parameters

Request parameters		Description
SubscriptionId		Id of the Subscription to add the items to. This id was returned from the CreateSubscription Service
TimestampsToReturn		OPC UA defines two timestamps, the source and the server timestamp. This parameter allows the client to define which timestamps the server should return with the value

(Continued)

ItemsToModify []		List of items to modify with the requested settings
	MonitoredItemId	Server defined handle for the Monitored Item used to identify the item in the Subscription. This id was returned from the CreateMonitoredItems Service
	ClientHandle	Client defined handle for the Monitored Item
	Monitoring Parameters	Client requested monitoring parameters sampling interval, filter, queue size, and discard policy
Response parameters		Description
Results []		List of results for the list of items to modify
	StatusCode	Status code that indicates if the modification of the Monitored Item succeeded
	Revised SamplingInterval	Sampling interval used by the server. If the requested rate is not available, the server adjusted the interval to the next available longer rate
	RevisedQueueSize	Queue size used by the server

5.7.3.4 SetMonitoringMode Service

This Service is used to set the monitoring mode for MonitoredItems in the Subscription. The parameters of the SetMonitoringMode Service are described in Table 5.37.

Table 5.37 SetMonitoringMode Service parameters

Request parameters		Description
SubscriptionId		Id of the Subscription containing the items to modify
MonitoringMode		Monitoring mode of the items which could be disabled, sampling, or reporting
MonitoredItemIds []		List of Monitored Items to modify. The ids were returned from the CreateMonitoredItems Service
Response parameters		Description
Results []		List of status codes for each item to modify indicating if setting the monitoring mode succeeded

5.7.3.5 SetTriggering Service

The Monitored Items Service Set allows adding items that are reported only when another item, the triggering item, triggers. This is done by creating links between the triggered items and the triggering item. The monitoring mode of the triggered items is set to sampling-only so that it will sample and queue notifications without reporting them. The parameters of the SetTriggering Service are described in Table 5.38.

Table 5.38 SetTriggering Service parameters

Request parameters	Description
SubscriptionId	Id of the Subscription containing the trigger items
TriggeringItemId	The server defined id for the Monitored Item that should be used as trigger item
LinksToAdd []	A list of server defined ids of Monitored Items that should be assigned to the triggering item
LinksToRemove []	A list of server defined ids of Monitored Items that should be removed from the triggering item
Response parameters	Description
AddResults []	List of status codes for each trigger link to add indicating if the add succeeded
RemoveResults []	List of status codes for each trigger link to remove indicating if the remove succeeded

5.7.4 Monitor Data Changes

Subscribing for data changes of Variable Values is the main use case in Classic OPC. OPC UA allows subscribing for more information than just Variable Value changes but it is expected that it is also one of the most important features of OPC UA. Table 5.39 describes the Value change specific MonitoredItem settings.

The other specific part is the structure of a notification message sent for a data change. The structure contains value, status of the value, the source timestamp, and the server timestamp assigned to the value. The same parameters are returned by the Read Service. They are described more detailed in Sect. 5.6.1.

It is expected that other Attributes than the Value Attribute or Properties of a Variable are changed very infrequently. Therefore the monitoring with a data Monitored Item should be avoided if possible. For this reason OPC UA provides two features used to monitor changes of metadata.

One feature is directly related to Values monitored with a data change Monitored Item. Any changes in the structure or the DataType of the sent data or any semantic changes of the Variable like a change of the engineering units are indicated in the normal data flow with the StructureChanged flag and the SemanticsChanged flag in the StatusCode assigned to each Value sent from the server to the client. If the StructureChanged flag is set, the client should read the data type information or the type description of a structured DataType. If the SemanticsChanged flag is set, the client should check the Properties of the Variable for changes.

Another feature is the SemanticsChange Event indicating changes in the Address Space semantic of the server. The changes covered by this Event are changes of Property Values. The AccessLevel Attribute of a Property indicates with the SemanticsChange bit if the Property is able to trigger such an Event if the Value is

changed. Servers will support this typically only for Properties where they get informed by the underlying system about such changes. Otherwise the server needs to monitor all Properties in the system if a client subscribes for such an Event which is even worse than monitoring Properties with a data change Monitored Item for changes.

Table 5.39 Data Monitored Item settings

Parameters		Description
NodeId		NodeId of the Variable to monitor
AttributeId		Value Attribute
SamplingInterval		The rate in milliseconds the server checks the underlying data source for changes. The type of change that triggers a notification is defined by the filter. If -1 is used for this interval, the publishing interval of the Subscription is used as for this setting. A client can over sample the Value by setting the SamplingInterval to a smaller value than the publishing interval and the queue size to 1
QueueSize		Maximum number of values stored for the Monitored Item during a publishing interval. After each publishing interval the server will send the values in the queue to the client
Filter		Data change filter settings
	Trigger	Type of change that triggers a notification message. The possible triggers are change in: • The status of the value • The value or status of the value (default) • The source timestamp, value, or status of the value
	DeadbandType	This parameter indicates if a deadband is applied and if applied, which type of deadband. OPC UA defines two type, absolute deadband and percent deadband
	DeadbandValue	<i>Absolute deadband</i> For this type the DeadbandValue contains the absolute change in a number data value that will cause a notification to be generated. Triggers a value change if $\text{abs}(\text{last value} - \text{new value}) > \text{DeadbandValue}$ <i>Percent deadband</i> For this type of deadband the DeadbandValue is defined as the percentage of the EURange. This deadband setting is only applied to Variables having a EURange Property. This setting triggers a value change if the value changed more than the percentage of the configured Value range

5.7.5 Monitor Events

The only way to receive current Events from a Server is the creation of an Event Monitored Item in a Subscription. Event and data Monitored Items can be combined in one Subscription. The main difference between an Event and a data Monitored Item is the way to select the subset of information to receive. For data items the client selects exactly one Variable Value and monitors the Value for changes. For an Event Item it is normally not possible to directly select the Event Source since Objects acting as EventNotifiers may combine a large number of Event Sources. For this reason the filter settings are very important to be able to reduce the amount of Events and Event Fields to the needed subset. The Event Monitored Item settings including the filter are described in Table 5.40.

Table 5.40 Event Monitored Item settings

Parameters		Description		
NodeId		NodeId of the Object to monitor		
AttributeId		EventNotifier Attribute		
SamplingInterval		0		
QueueSize		0 – this means that the maximum size supported by the server is used		
Filter		Event filter settings		
	SelectClauses []		List of select clauses used to select the Event fields to return for each Event notification	
		TypeId	NodeId of the Event Type defining the Event field	
		BrowsePath []	List of BrowseNames from the Event Type to the Instance Declaration representing the Event field. This list has one element for simple Event Types. If the list has more elements the server must follow forward hierarchical references to find the Event field	
	WhereClause		Limits the notifications to those Events that match the criteria defined by this ContentFilter	
		Elements []		List of operators and their operands that compose the filter criteria. The filter is evaluated by starting with the first entry in this array
			FilterOperator	Filter operator to be evaluated. Possible operators are described in Table 5.41
			Filter Operands []	Array of extensible parameters containing the operands used by the selected operator. The number and use depend on the operands described in Table 5.41. This array needs at least one entry

The *Select* clause is used to reduce the number of data contained in an Event notification. The client must explicitly select Event fields he is interested in. There are no default fields returned like in Classic OPC.

The *Where* clause is used to filter on the Event fields to reduce the number of Events to the ones the client is interested in. Examples for typical filters are:

- EventType = TransitionEventType
- ((SourceNode = DeviceX) OR (SourceNode = DeviceY)) AND (Severity > 200)
- (EventType = MyEventType) AND (Severity > 500)

Figure 5.11 shows the fields of the BaseEventType, a derived Event Type MyEventType, and an Event filter with a select of some fields of both Types and a Where clause using other fields to filter the Events.

The operators and operands are defined by OPC UA. The operators are described in Table 5.41. Additional information can be found in [UA Part 4]. The Operands could be:

- A index of another element in the list used to build a logic tree of elements
- A literal value, e.g., the NodeId of the EventType to filter for
- An operand⁶ identifying the field of the new Event used to filter on, e.g., the EventType field defined by the BaseEventType.

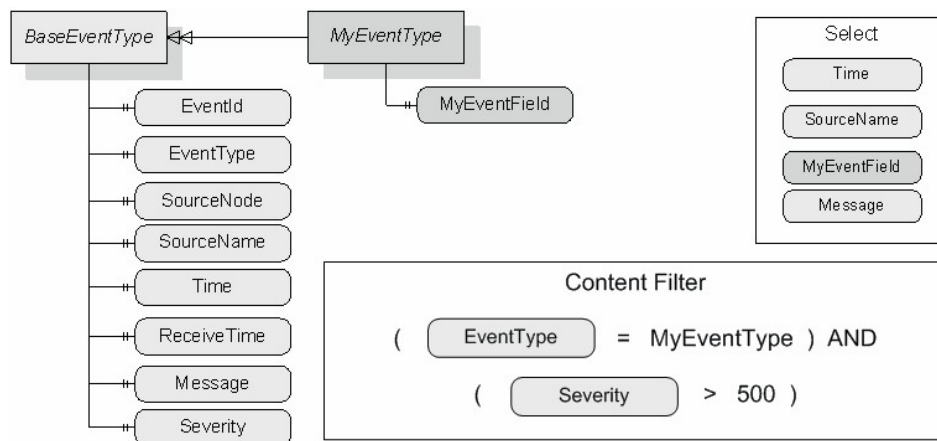


Fig. 5.11 Example for Event filter

Table 5.41 Filter operators

Operator	Description
Equals	True if operand one is equal to operand two
IsNull	True if operand one is null
GreaterThan	True if operand one is greater than operand two
LessThan	True if operand one is less than operand two
GreaterThanOrEqual	True if operand one is greater or equal to operand two

(Continued)

⁶SimpleAttributeOperand containing similar information like a SelectClause element.

LessThanOrEqual	True if operand one is less or equal to operand two
Like	True if operand one matches a pattern defined by operand two. The pattern syntax is defined in [UA Part 4]
Not	True if operand one is false
Between	True if operand one is greater or equal to operand two and less than or equal to operand three
InList	True if operand one is equal to one or more of the remaining operand
And	True if operand one AND operand two are true
Or	True if operand one OR operand two are true
Cast	Converts operand one to a data type identified with a DataType NodeId in operand two

5.7.6 Monitor Aggregated Data

A special version of subscribing for data changes of Variable Values is the monitoring of aggregated data where the server samples on a higher rate, calculates the selected aggregate after the SamplingInterval and sends only the aggregated values to the client. Table 5.42 describes the aggregate specific MonitoredItem settings. More details about aggregate calculation are described in Chapter 5.9.1 in the section for HistoryRead processed and in [UA Part 13].

Table 5.42 Aggregate MonitoredItem settings

Parameters		Description
NodeId		NodeId of the Variable to monitor
AttributeId		Value Attribute
SamplingInterval		The interval in milliseconds for which the server calculates the aggregate
QueueSize		Maximum number of values stored for the Monitored Item during a publishing interval. After each publishing interval the server will send the values in the queue to the client
Filter		Aggregate filter settings
	StartTime	Start time of the first interval to calculate. The length of the intervals is defined by the SamplingInterval
	AggregateType	NodeId of the aggregate type. The list of aggregates is described in Table 5.48
	RawData SamplingInterval	The rate at which values are sampled from the underlining system to be used to compute the aggregate

The structure of a notification message sent for an aggregate result is the same like for data MonitoredItems.

5.8 Calling Methods Defined by the Server

OPC UA allows servers to expose Methods in the Address Space that can be called by clients. Methods are components of Objects and can be called in the context of an Object. All necessary information to call a Method is available in the Method description including the detailed description of the input and the output parameters of the Method. Section 5.5.2.3 describes how to find this information in the use case to get information to call a method.

The Call Service is used to actually call a method with built-in knowledge of the client or based on knowledge the client got through the information available in the Address Space. The Service allows calling a list of Methods to reduce the roundtrips between the client and the server. OPC UA client SDKs will provide simplified functions to call one Method and full featured functions to call a list of Methods in one Service invocation. The parameters of the Call Service are described in Table 5.43.

Table 5.43 Call Service parameters

Request parameters		Description
MethodsToCall []		List of Methods to call
	ObjectId	NodeId of the Object or ObjectType Node that provides the Method
	MethodId	NodeId of the Method to call in the context of the Object or ObjectType
	InputArguments []	List of input argument values of the Method. The required arguments and the necessary data types for each argument are defined by the InputArgument Property of the Method in the Address Space
Response parameters		Description
Results []		List of results for each Method to call
	StatusCode	Success code for the Method call
	InputArgument Results []	List of input argument result codes for the Method call
	OutputArguments[]	List of output argument values of the Method. The returned arguments and the data types for each argument are defined by the OutputArgument Property of the Method in the Address Space

5.9 Access History of Data and Events

The main difference between the access to current data and Events using Read, Write, and Subscriptions and the access to the history is the definition of a time domain in the history request and the return of an array of information archived for the time period domain of the current data or Events. The availability of history is indicated by the AccessLevel Attribute of a Variable or the EventNotifier Attribute of an Object.

The history access Services HistoryRead and HistoryUpdate are making extensive use of extensible parameters to cover the different use cases for history access with two Services. [UA Part 4] defines only the fixed Service parameters. All extensible parameters are defined in [UA Part 11].

The HistoryRead Service request uses an extensible parameter to define the type of read for raw data, modified data, processed data, data on specific timestamps, and history of Events. The HistoryRead response is using an extensible parameter for the transport of the two types of requested information, data and Events. The HistoryUpdate Service request uses an extensible parameter to insert, to replace, to update, and to delete data or Events.

To describe these different variations of the Services, the following sections are describing first the Service with the common parameters and then the different types of extensible parameters. OPC UA client SDKs will typically expose nine history methods. Their signatures are defined by the different types of extensible parameters.

5.9.1 *HistoryRead Service*

This Service is used to read historical Values or Events of one or more Nodes in an ordered sequence for the defined time domain. Continuation points are used to continue the read of the ordered sequence if not all data can be returned in one HistoryRead response. The returned number can be limited by the client or the server. Table 5.44 describes the general HistoryRead Service parameters. The extensible parameters for the different types of history Read are described in the following Sections.

Table 5.44 General HistoryRead Service parameters

Request parameters		Description
HistoryReadDetails		Extensible parameter containing the parameters for the different types of history read operations. This could be a read of raw, modified, and processed values, read of Values at certain timestamps or the read of Event history
TimestampsToReturn		Indicates if the source timestamp, the server timestamp, or both should be returned The selected timestamp is also used for the selection of the values in the time domain to read. If both are selected, the source timestamp is used
Release ContinuationPoint		The flag indicates if the Service call is used to release ContinuationPoints returned from previous calls without returning additional data. This allows clients to free resources in the server if the client does not continue the read
NodesToRead []		List of Nodes where the client wants to read history
	NodeId	NodeId of the Node that provides historical information For reading data history this must be the NodeId of a Variable Node where the HistoryRead flag is set for the Access Level Attribute For reading Event history this must be the NodeId of an Object Node where the HistoryRead flag is set for the EventNotifier Attribute
	ContinuationPoint	Returned by the server in a previous HistoryRead call. It is used by the client to continue the read or to release the continuation point
Response parameters		Description
Results []		List of results for each Node to read
	StatusCode	Success code for the Node and the filter settings
	HistoryData	Extensible parameter containing the result data for the different types of history read operations. This could be a list of data values or a list of Event notifications
	ContinuationPoint	Set by the server if not all data can be returned in this call. This allows the client to continue the read without exceeding the limits set by the client or the server

5.9.1.1 Reading Raw or Modified Data

The HistoryRead Service is called with the extensible parameter of type Read-RawModified to read raw or modified data for a specified time domain. It reads

the Values, status, and timestamps of one or more Variables. The extensible parameter is described in Table 5.45 and is used in the HistoryReadDetails parameter of the HistoryRead Service.

For raw data the values stored in the history database are returned directly. A modified value is a value that has been replaced by another value at the same timestamp in the history database. If there are multiple replaced values the server must return all of them.

Table 5.45 ReadRawModified extensible parameter

Parameters	Description
IsReadModified	Specifies the type of read. If the flag is set to false, a read raw is performed, if set to true a read modified is performed
StartTime	Begin of the time period to read
EndTime	End of the time period to read
NumValuesPerNode	The maximum number of values returned for each Node. If this maximum is exceeded by the number of available values in the defined time domain, the server returns a continuation point. The value 0 indicates that there is no limit set by the client. The server can also reduce the number of values to return
ReturnBounds	A flag that indicates if bounding values should be returned. If set to true, a value before the start time and a value after the end time is returned if no values are available at the specified start and end timestamps

The extensible parameter type HistoryData is used to return the data for read raw, modified, processed and read at time. The content of the HistoryData structure is described in Table 5.46. The data value structure is also used in the Read Service and data change notifications. The parameters are described more detailed at the Read Service in Table 5.25.

Table 5.46 HistoryData extensible parameter

Parameters	Description
DataValues []	Array of data values containing the result data
Value	Raw or processed value from the history database
StatusCode	Status of the value. There are special historical access status codes and info bits defined providing history specific information
SourceTimestamp	Source timestamp for the value
ServerTimestamp	Server timestamp for the value

5.9.1.2 Reading Processed Data

The HistoryRead Service is called with the extensible parameter of type ReadProcessed to Read processed data calculated with the specified aggregate based on the raw data in the history database. It reads the processed Values, status and time-stamps for one or more Variables in the specified time domain. The extensible parameter is described in Table 5.47 and is used in the HistoryReadDetails parameter of the HistoryRead Service. The server must use start time, end time, and the resample interval to generate a sequence of time intervals and then calculate an aggregate for each interval. The aggregates are defined in [UA Part 13] and described in Table 5.48.

Table 5.47 ReadProcessed extensible parameter

Parameters	Description
StartTime	Begin of the time period to read
EndTime	End of the time period to read
ResampleInterval	Time interval in milliseconds that is used to calculate one aggregated value from the raw values in the history database. The time domain is divided into sub-intervals with the length of the ResampleInterval beginning with the start time If the ResampleInterval is 0, one aggregated value is calculated for the time domain
AggregateType	The NodeId of the aggregate used for the calculation of the values. The OPC UA defined aggregates are described in Table 5.48

All aggregates in the following table without an additional comment return a timestamp of the start of the interval. If the aggregate returns another timestamp, the specific behavior is described for the aggregate.

Table 5.48 Aggregates used for HistoryRead and monitoring of aggregated values

Aggregate	Description
Interpolative	Returns an interpolated value for the starting time of the interval
Data averaging and summation aggregates	
Average	Returns the average data over the resample interval. It adds up all values in the interval and divides the sum by the number of values
TimeAverage	Returns the time weighted average data over the resample interval. A straight line is drawn between each raw value in the interval. The area under the line is divided by the length of the interval to yield the average. Interpolated values are used at the start and at the end of the interval

(Continued)

Total	Returns the sum of the data over the resample interval. It adds up all values in the interval
Totalize Average	Returns the totalized Value (time integral) of the data over the resample interval
Data variation aggregates	
Minimum	Returns the minimum value in the resample interval
Maximum	Returns the maximum value in the resample interval
Minimum ActualTime	Returns the minimum value in the resample interval and the timestamp of the minimum value
Maximum ActualTime	Returns the maximum value in the resample interval and the timestamp of the maximum value
Range	Returns the difference between the minimum and maximum Value over the sample interval
Counting aggregates	
Annotation Count	Returns the number of annotations in the resample interval. Annotations are user entered comments in the history database
Count	Returns the number of raw values with good status in the resample interval
Duration InState0	The duration of the interval the value was in FALSE state. The value of the aggregate is the time in milliseconds
Duration InState1	The duration of the interval the value was in TRUE state. The value of the aggregate is the time in milliseconds
Number OfTransitions	Returns the number of value changes with good quality in the resample interval
Time aggregates	
Start	Returns the first value in the interval and the timestamp of the first value
End	Returns the last value in the interval and the timestamp of the last value
Delta	The difference between the first and the last good value in the interval. If the last value is less than the first value, the result will be negative
Data quality aggregates	
DurationGood	The duration of the interval the value had good quality. The value of the aggregate is the time in milliseconds
DurationBad	The duration of the interval the value had bad quality. The value of the aggregate is the time in milliseconds
PercentGood	Percentage of the interval the value had good quality (0–100). PercentGood = DurationGood / ResampleInterval * 100
PercentBad	Percentage of the interval the value had bad quality (0–100). PercentGood = DurationGood / ResampleInterval * 100
WorstQuality	Returns the worst quality of the values in the interval. The value of the aggregate is the status code for the worst quality

5.9.1.3 Reading Data at a Series of Timestamps

The HistoryRead Service is called with the extensible parameter of type ReadAtTime to read data for the specified timestamps. When no value exists for a specified timestamp, a value is interpolated from the surrounding values to represent the value at the specified timestamp. It reads the Values, status, and timestamps for one or more Variables. The extensible parameter is described in Table 5.49 and is used in the HistoryReadDetails parameter of the HistoryRead Service.

Table 5.49 ReadAtTime extensible parameter

Parameters	Description
RequestedTimes []	List of requested timestamps

5.9.1.4 Reading Event History

The HistoryRead Service is called with the extensible parameter of type ReadEvent to read Events for the specified time domain. The filter parameter is used to determine which historical Events are returned and selects the Event field returned for an Event. The extensible parameter is described in Table 5.50 and is used in the HistoryReadDetails parameter of the HistoryRead Service.

Table 5.50 ReadEvent extensible parameter

Parameters	Description
NumValuesPerNode	The maximum number of Events returned for each Node. If this maximum is exceeded by the number of available events in the defined time domain, the server returns a continuation point. The value zero indicates that there is no limit set by the client. The server can also reduce the number of events to return
StartTime	Begin of the time period to read
EndTime	End of the time period to read
Filter	The filter allows reducing the amount of Events returned from the Read by using the same Event filter used for monitoring current Events. The filter is described there in Table 5.40

The extensible parameter type HistoryEvent is used to return the Events for Read Events. The content of the HistoryEvent structure is described in Table 5.51.

Table 5.51 HistoryData extensible parameter

Parameters	Description
Events []	Array of Events being delivered
EventFields []	List of selected Event fields

5.9.2 HistoryUpdate Service

This Service is used to insert, replace, update, or delete historical Values or Events. Table 5.52 describes the general HistoryUpdate Service parameters. The extensible parameters for the different types of history update are described in the following sections.

Table 5.52 General HistoryUpdate service parameters

Request parameters	Description
HistoryUpdate Details []	A list of extensible parameters containing the information for the different types of history updates. This could be insert, replace, update, or delete of historical Values or Events
Response parameters	Description
Results []	List of results for each update detail list entry
StatusCode	Status code for the list entry
OperationResults []	Status code for each operation in the list entry

5.9.2.1 Insert, Replace, or Update Data

The HistoryUpdate Service is called with the extensible parameter of type UpdateData to insert, replace, or update data in the history database. The extensible parameter is described in Table 5.53 and is used in the HistoryUpdateDetails parameter of the HistoryUpdate Service.

Table 5.53 UpdateData extensible parameter

Parameters	Description
PerformInsert	A flag that indicates if an insert should be performed if no value is available for the passed timestamp in the history database
PerformReplace	A flag that indicates if a replace should be performed if a value is available for the passed timestamp in the history database
NodeId	NodeId of the Variable to be updated
DataValues []	A list of values including status codes and timestamps to update in the history database

One of the two flags to force insert or replace must be set. Table 5.54 describes the update Event flags usage.

Table 5.54 Update data flags

Insert	Replace	Description
True	False	The passed value will only be written to the history if no value exists at the specified timestamp
False	True	The passed value will only be written to the history if a value exists at the specified timestamp. The existing value will be replaced
True	True	The passed value will be inserted if no value exists for the timestamp but will also replace an existing value at the given timestamp

5.9.2.2 Insert, Replace, or Update Event

The HistoryUpdate Service is called with the extensible parameter of type UpdateEvent to insert, replace, or update, Events in the history database. The extensible parameter is described in Table 5.55 and is used in the HistoryUpdateDetails parameter of the HistoryUpdate Service.

Table 5.55 UpdateEvent extensible parameter

Parameters	Description
PerformInsert	A flag that indicates if an insert should be performed if no event is available for the passed timestamp in the history database
PerformReplace	A flag that indicates if a replace should be performed if an event is available for the passed timestamp in the history database
NodeId	NodeId of the Object to be updated
Filter	The filter is used to find the event to update or to replace or to insert the Event if PerformInsert is set and the Event was not found. The same Event filter is used like for monitoring current Events. The filter is described there in Table 5.40
EventFields []	List of Event fields for the Event to update

One of the two flags to force insert or replace must be set. Table 5.56 describes the update Event flags usage.

Table 5.56 Update Event Flags

Insert	Replace	Description
True	False	The passed Event will only be written to the history if no Event exists at the specified timestamp
False	True	The passed Event will only be written to the history if an Event exists at the specified timestamp. The existing Event will be replaced
True	True	The passed Event will be inserted if no Event exists for the timestamp but will also replace an existing Event at the given timestamp

5.9.2.3 Delete Raw or Modified Data

The HistoryUpdate Service is called with the extensible parameter of type DeleteRawModified to delete raw or modified data for a specified time domain. The extensible parameter is described in Table 5.57 and is used in the HistoryUpdateDetails parameter of the HistoryUpdate Service.

For raw the values stored in the history database are deleted. A modified value is a value that has been replaced by another value at the same timestamp in the history database. If there are multiple replaced values the server must delete all of them.

Table 5.57 DeleteRawModified extensible parameter

Parameters	Description
IsDeleteModified	Specifies the type of delete. If the flag is set to false, a delete raw is performed, if set to true a delete modified is performed
NodeId	NodeId of the Variable for which the values are to be deleted
StartTime	Begin of the time period to delete
EndTime	End of the time period to delete

5.9.2.4 Delete Data at a Series of Timestamps

The HistoryUpdate Service is called with the extensible parameter of type DeleteAtTime to delete data for the specified timestamps. The extensible parameter is described in Table 5.58 and is used in the HistoryUpdateDetails parameter of the HistoryUpdate Service.

Table 5.58 DeleteAtTime extensible parameter

Parameters	Description
NodeId	NodeId of the Variable for which the values are to be deleted
RequestedTimes []	List of requested timestamps

5.9.2.5 Delete Events

The HistoryUpdate Service is called with the extensible parameter of type DeleteEvent to delete specific events. The extensible parameter is described in Table 5.59 and is used in the HistoryUpdateDetails parameter of the History-Update Service.

Table 5.59 DeleteEvent extensible parameter

Parameters	Description
NodeId	NodeId of the Object for which the events are to be deleted
EventId []	An array of Event ids to identify which Events are to be deleted

5.10 Find Information in Complex Address Space

The Browse Service is used to navigate through the Address Space. This mechanism is useful to navigate through known areas and to find information in a small Address Space but it will be difficult or impossible to find the necessary information in a very large or a very dynamic Address Space. Address Spaces of servers providing access to systems with rich Information Models can have millions of Nodes. Address Spaces of MES or ERP systems can have very dynamic Address Spaces representing for example current work orders.

The OPC UA feature used to find information in such Address Spaces is Query. It implements a different approach than Browse. It allows defining filter criteria to retrieve a subset of Nodes and Information based on these filters, whereas Browse defines a starting Node and filters to reduce the list of returned referenced Nodes. The query mechanism of OPC UA is based on type information. The starting point of each query is to specify the type of Objects or Variables the client is interested in. In the SELECT-Part (DataToReturn) of a query, the client specifies what data should be returned relative to instances of the type and in the WHERE-Part (Filter) the client specifies the filter criteria.

The QueryFirst Service is used to start a Query and QueryNext is used to get the remaining results if too many results are available. The Service parameters of QueryFirst are described in Table 5.60.

Table 5.60 QueryFirst Service parameters

Request parameters		Description
View		Passing a View allows limiting the Query to a specific View
NodeTypes []		Array of structures containing the type filter for the Query
	TypeDefinitionNode	NodeId of the TypeDefinition VariableType or ObjectType where instances should be returned
	IncludeSubtypes	Indicates whether instances of subtypes should be included
	DataToReturn []	Specifies the Nodes and the Attributes of the instances to return
	RelativePath	Brows Path from the instance of the specified type to the component of the instance
	AttributeId	Id of the Attribute to return for the selected Node
Filter		Content filter used to reduce the returned instance Nodes and their Attribute values. The content filter is the same filter that is used in the where clause of the Event Monitored Items. The filter settings are described there in Tables 5.40, 5.41
MaxDataSetToReturn		This parameter allows the client to limit the number of returned results
Response parameters		Description
QueryDataSets []		List of result data
	NodeId	NodeId of the instance Node matching the query defined in the request
	TypeDefinitionNode	TypeDefinition NodeId for the returned instance Node
	Values []	List of Values for the selected Attribute
ContinuationPoint		A continuation point is returned when the server was not able to deliver all results in the QueryFirst response. The limitation can be set by the client in the request or by the server during Query processing The continuation point can be passed to QueryNext to get the remaining results QueryNext takes the ContinuationPoint as input parameter and returns the same results like QueryFirst

5.11 Modify the Address Space

The NodeManagement Services enables an OPC UA client to create and delete Nodes and References in the Address Space of an OPC UA server. It is expected that this feature is mainly used between OPC UA servers and their configuration

clients. Information Models may define the use of these Services more detailed to make them useful for generic clients knowing the Information Model.

5.11.1 Adding Nodes

The AddNodes Service is used to add one or more Nodes to the Address Space. The parameters of the Service are described in Table 5.61.

Table 5.61 AddNodes Service parameters

Request parameters		Description
NodesToAdd []		List of Nodes to add to the server Address Space
	ParentNodeId	NodeId of the Node where the new Node should be referenced from
	ReferenceTypeId	NodeId of the Reference type used for the Reference to create between the parent Node and the new Node
	BrowseName	Browse name of the new Node
	NodeClass	Node class of the new Node
	NodeAttributes	Extensible parameter containing the additional Attribute values depending on the NodeClass to create
	TypeDefinition	TypeDefinition NodeId used to define which type of Object or Variable with its type defined components should be created. This parameter is not set if the NodeClass is not Object or Variable
Response parameters		Description
Results []		List of results for each add operation
	StatusCode	Result code for the add operation
	AddedNodeId	NodeId of the newly created Node

5.11.2 Creating References Between Nodes

The AddReferences Service is used to create one or more References between Nodes in the Address Space. The parameters of the Service are described in Table 5.62.

Table 5.62 AddReferences Service parameters

Request parameters		Description
ReferencesToAdd []		List of References to create
	SourceNodeId	NodeId of the source Node of the new Reference
	ReferenceTypeId	NodeId of the Reference type used for the Reference to create between the source and the target Node
	TargetNodeId	NodeId of the target Node of the new Reference
Response parameters		Description
Results []		List of result codes for each add operation

5.11.3 Removing Nodes

The DeleteNodes Service is used to remove one or more Nodes from the Address Space. The parameters of the Service are described in Table 5.63.

Table 5.63 DeleteNodes Service parameters

Request parameters		Description
NodesToDelete []		List of Nodes to remove from the server Address Space
	NodeId	NodeId of the Node to delete
	DeleteTarget Reference	A flag that indicates if the server should also delete the Reference to the target Node
Response parameters		Description
Results []		List of result codes for each delete operation

5.11.4 Delete References Between Nodes

The DeleteReferences Service is used to remove one or more References in the Address Space. The parameters of the Service are described in Table 5.64.

Table 5.64 DeleteReferences Service parameters

Request parameters		Description
ReferencesToDelete []		List of References to delete from the server Address Space
	SourceNodeId	NodeId of the source Node of the Reference to delete
	ReferenceTypeId	NodeId of the Reference type used for the Reference to delete between the source and the target Node
	TargetNodeId	NodeId of the target Node of the Reference to delete
Response parameters		Description
Results []		List of result codes for each delete operation

5.12 Summary

5.12.1 Key Messages

There are two types of OPC UA Services. One type is used to create a communication context like the Secure Channel, Session, Subscription, and MonitoredItem Service Sets. The other type is used to exchange information like Browse to get information about the structure of the Address Space, Read, Write, and Publish to access data and Call to execute Methods.

Compared to Classic OPC the number of Services⁷ is reduced to a small set based on the design goal to provide only one generic Service for a task and on the design goal to provide information by modeling the information and not by providing a special access method for the type of information.

The Services are defined in an abstract way enabling the implementation of the Service transport with different technologies and different development environments to achieve the requirements for platform-independence, scalability, and high performance but also for Internet access and the capability to cross firewalls.

All Services can be called synchronous and asynchronous since the base mechanism to exchange the messages is asynchronous and the calls to the client API can be made synchronous in the UA Stack. Individual timeouts per Service call can be configured. A communication context like a Session or a Subscription can still be used by the client after short network interruptions but the negotiated timeouts are allowing the server to clear resources after the timeout expires. Together with the Publish mechanisms to ensure no data gets lost and the built-in redundancy features (see Sect. 9.3) a reliable communication between distributed systems is ensured.

5.12.2 Where to Find More Information?

The abstract OPC UA Services are defined in [UA Part 4]; the technology mapping is defined in [UA Part 6].

Additional implementation specific information is provided by the documentation of the different UA Stack implementations and OPC UA SDKs.

5.12.3 What's Next?

After explaining the abstract Services to access the OPC UA information provided by a server, Chap. 6 describes the different technology mappings of the Services to the different types of message serialization, message security, and message transport and the different available implementations of these mappings.

Other related aspects are the Security concepts described in Chap. 7, the Application Architecture explained in Chap. 8 and the System Architecture described in Chap. 9.

⁷Comparable to interface methods in COM.